

教材與每日課程說明

本文件依照課程日期，彙整每一堂課的學習目標、核心概念說明、實作步驟、程式碼範例（虛擬碼）與課後作業，方便學員依日期查閱當週教材。

課程總覽

日期	主題
7/21	第01堂 AI原生開發與LLM核心架構
7/28	第02堂 AI Agent 邏輯建構與自動化引擎配置
8/04	第03堂 巨量數據調度工程與非結構化資料轉換
8/11	第04堂 跨平台 Web 數據擷取與異質系統整合
8/18	第05堂 智慧通訊協作：電子郵件與 Office 流程自動化
8/25	第06堂 檔案系統運維與傳統系統介面整合
9/01	第07堂 Vibe Coding 驅動：智慧動態報表系統開發
9/08	第08堂 實作：智慧多通路內容行銷與分發系統
9/15	第09堂 實作：AI 智慧研報與商務情報搜集系統
9/22	第10堂 實作：AI 智慧驅動視覺化簡報生成
9/29	第11堂 實作：智慧客服郵件決策支援系統
10/06	第12堂 進階實務：Vibe Coding 與 Agent 系統協作
10/13	第13堂 案例分析：企業供應鏈數據管理與結構化工程
10/20	第14堂 案例分析：敏捷管理自動化—智慧週報生成系統

目錄

- 1. 7/21 | 第01堂 | AI原生開發與LLM核心架構
- 2. 7/28 | 第02堂 | AI Agent 邏輯建構與自動化引擎配置
- 3. 8/04 | 第03堂 | 巨量數據調度工程與非結構化資料轉換
- 4. 8/11 | 第04堂 | 跨平台 Web 數據擷取與異質系統整合
- 5. 8/18 | 第05堂 | 智慧通訊協作：電子郵件與 Office 流程自動化
- 6. 8/25 | 第06堂 | 檔案系統運維與傳統系統介面整合
- 7. 9/01 | 第07堂 | Vibe Coding 驅動：智慧動態報表系統開發
- 8. 9/08 | 第08堂 | 實作：智慧多通路內容行銷與分發系統
- 9. 9/15 | 第09堂 | 實作：AI 智慧研報與商務情報搜集系統
- 10. 9/22 | 第10堂 | 實作：AI 智慧驅動視覺化簡報生成
- 11. 9/29 | 第11堂 | 實作：智慧客服郵件決策支援系統
- 12. 10/06 | 第12堂 | 進階實務：Vibe Coding 與 Agent 系統協作
- 13. 10/13 | 第13堂 | 案例分析：企業供應鏈數據管理與結構化工程

7/21 | 第01堂 | AI原生開發與LLM核心架構

學習目標

完成本堂課後，學員將能夠：

- 說明大語言模型（LLM）的基本原理與主要產業應用場景
- 描述 AI Agent 的系統架構與四大核心組成
- 體驗 Vibe Coding 的開發方式，並理解其長處與限制
- 認識 OpenClaw 平台的基礎概念
- 識別使用 LLM 所衍生的基本資安風險

核心概念說明

1. 大語言模型（Large Language Model, LLM）

大語言模型是一種以海量文字資料訓練而成的 AI 系統，能夠理解並生成自然語言。

一個簡單的比喻：想像有一位助理，他從小閱讀了幾乎所有的書籍、文章、程式文件與對話紀錄。當你給他一個問題或指令，他會根據所有學過的知識生成一個最合理的回應。這就是 LLM 的運作方式。

常見的 LLM 產品：

產品	開發商	特點
GPT-4o	OpenAI	業界廣泛使用的通用模型
Claude	Anthropic	強調安全性與長文本處理
Gemini	Google	整合 Google 服務生態
Llama 3	Meta	開源模型，可在本地部署

LLM 的核心能力：

- **文字生成（Text Generation）：**根據輸入的 Prompt 產出文字內容
- **語意理解（Semantic Understanding）：**理解語境與隱含意圖，不只是字面意思
- **指令遵循（Instruction Following）：**根據具體指令執行特定任務
- **推理與分析（Reasoning）：**進行多步驟的邏輯推論

LLM 在產業的實際應用：

- 客服自動回覆：全天候回應客戶常見問題
- 文件摘要與分析：快速萃取大量報告的關鍵資訊
- 程式碼自動生成：加快軟體開發速度
- 行銷文案量產：針對不同受眾快速生成內容
- 資料分類與標記：自動整理非結構化資料

2. AI Agent 系統架構

AI Agent (AI 代理) 是一種能夠**自主執行複雜任務**的 AI 系統。與普通的 LLM 對話不同, Agent 具有完整的「感知 → 思考 → 規劃 → 行動」迴圈。

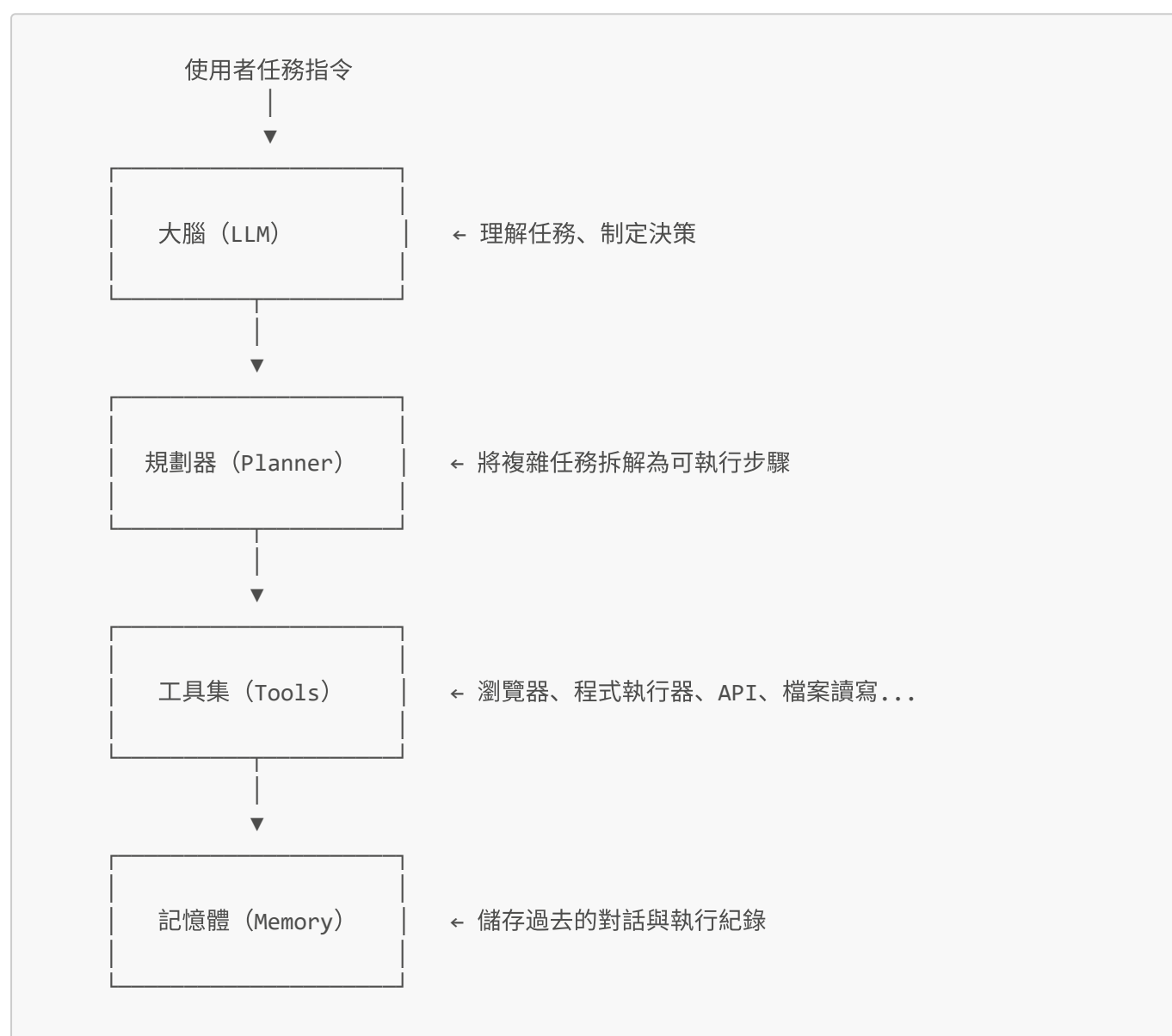
直覺的比喻:

想像你雇用了一位助理, 你告訴他:「幫我調查三個競爭對手的定價, 整理成 Excel 報表後寄給我。」

- 助理會**自行規劃步驟**: 搜尋網路 → 瀏覽各官網 → 紀錄資料 → 開啟 Excel → 填寫資料 → 寄送郵件
- 全程不需要你一步一步指示, 他會自己完成

AI Agent 就是能夠**自主規劃並執行多步驟任務**的 AI 系統。

Agent 的四大組成要素:



3. Vibe Coding

Vibe Coding 是一種以**自然語言驅動**程式碼生成的開發方式。你不需要自己寫每一行程式語法, 只需要用白話文描述你想要做的事, 由 AI 工具 (如 GitHub Copilot) 幫你生成可執行的程式碼。

Vibe Coding 的長處：

- 大幅降低程式設計的學習門檻
- 快速將想法轉化為可執行的自動化工具
- 讓非技術背景的業務人員也能構建流程
- 加快原型（Prototype）開發速度

Vibe Coding 的限制：

- AI 生成的程式碼可能包含錯誤，**必須人工驗證**
- 對非常複雜或高安全需求的系統有一定侷限
- 若描述需求不清楚，AI 可能生成不符合期望的程式碼
- 不適合直接用於需要高精確度的核心系統

傳統開發 vs. Vibe Coding 比較：

	傳統開發	Vibe Coding
前置要求	熟悉程式語法與演算法	能清楚描述需求
開發速度	依工程師熟練度而定	快速產出原型
結果可靠性	工程師掌控	需人工驗證
適合場景	核心系統、高安全性	自動化工具、快速驗證

4. OpenClaw 平台簡介

OpenClaw 是目前產業廣泛採用的 AI Agent 視覺化開發平台，讓使用者以**拖拉積木**的方式組合 AI 工作流程，大幅降低 Agent 系統的開發難度。

OpenClaw 的主要特色：

- 直覺的視覺化流程編輯介面
- 支援串接多種 LLM 後端（GPT、Claude、Llama 等）
- 內建豐富的工具節點：網頁搜尋、讀取檔案、呼叫 API、資料庫操作等
- 支援多個 Agent 相互協作完成複雜任務

本課程後續堂次將會實際使用 OpenClaw 構建各種自動化 Agent 系統。

5. LLM 與 Agent 的資安風險

在學習 AI Agent 開發的同時，我們也必須了解其潛在的資安風險，才能在設計系統時做出正確的防護措施。

主要風險類型：

風險類型	說明	防護方向
Prompt Injection	惡意使用者透過精心設計的輸入，操控 AI 執行非預期的行為	對外部輸入進行驗證與過濾

風險類型	說明	防護方向
資料洩漏	Agent 在執行任務時可能不慎將敏感資料傳送至外部 LLM	避免將機密資料送入外部 API
過度授權	Agent 被賦予超出任務所需的系統權限	遵循「最小權限原則」
幻覺 (Hallucination)	LLM 生成看似合理但實際錯誤的資訊	重要輸出需人工核實

重要觀念： LLM 因為訓練方式的特性，有時會「自信地給出錯誤答案」。這在 Agent 系統中尤其危險，因為錯誤的決策會直接驅動工具執行行動。設計 Agent 時，務必在關鍵決策點加入人工確認機制。

實作步驟（課堂示範）

本堂課由教師示範操作，學員跟隨觀察並理解整體概念。

示範一：安裝開發環境

步驟 1：安裝 Python

- 1. 前往 <https://www.python.org/downloads/> 下載最新版本
- 2. 安裝時勾選「Add Python to PATH」選項

步驟 2：安裝 Visual Studio Code

- 1. 前往 <https://code.visualstudio.com/> 下載安裝
- 2. 完成後開啟 VS Code

步驟 3：安裝 GitHub Copilot

- 1. 在 VS Code 左側擴充套件搜尋欄輸入 **GitHub Copilot**
- 2. 點選安裝
- 3. 使用 GitHub 帳號登入並完成授權

步驟 4：測試 Copilot 正常運作

- 1. 點選左側 Copilot 圖示開啟 Chat 視窗
- 2. 輸入「你好，請用中文自我介紹」測試回應

示範二：用 Vibe Coding 生成第一個 AI 程式

步驟 1：建立工作資料夾 在桌面建立一個名為 **ai-course** 的資料夾，並用 VS Code 開啟（檔案 → 開啟資料夾）。

步驟 2：在 Copilot Chat 輸入需求 在 Copilot Chat 視窗輸入以下描述：

請幫我用 Python 寫一個程式，讓我在終端機輸入文字，
程式會呼叫 OpenAI 的 API，並把 AI 的回應顯示在螢幕上。

需要支援連續對話，輸入 "exit" 就結束程式。
請記得從環境變數讀取 API Key，不要把 Key 寫在程式碼裡。

步驟 3：觀察生成的程式碼結構

觀察 Copilot 生成的 Python 程式，注意理解以下部分：

- **API Key 的讀取方式**：為何不該把 Key 直接寫在程式碼裡？
- **對話歷史的儲存方式**：為何需要保存每一輪的對話？
- **請求與回應的流程**：程式如何把你的輸入傳給 AI 並接收回應？

步驟 4：加入工具調用功能

教師將進一步修改程式，加入一個「讀取/寫入檔案」的工具，讓這個程式能夠操作電腦上的文字檔案，示範 Agent 的基本概念：當 AI 有了「工具」，它就從一個「聊天機器人」升級為「可以做事的代理人」。

程式碼範例（虛擬碼）

基礎 LLM 對話程式

```
# === 初始化 ===  
載入 openai 套件  
設定 API_KEY = 從環境變數讀取("OPENAI_API_KEY")  
  
# === 初始化對話歷史 ===  
對話歷史 = []  
  
# === 主要對話迴圈 ===  
重複執行：  
    讀取 使用者輸入  
  
    如果 使用者輸入 == "exit":  
        顯示 "再見！"  
        結束程式  
  
    將「使用者訊息」加入對話歷史  
  
    呼叫 LLM API：  
        傳入：[模型名稱, 對話歷史]  
        接收：AI 回應文字  
  
    將「AI 回應」加入對話歷史  
    顯示 AI 回應
```

加入工具調用的基礎 Agent 機制

```
# === 定義可用工具 ===  
工具A: 讀取檔案(檔案路徑) → 回傳檔案內文字
```

工具B：寫入檔案(檔案路徑，文字內容) → 儲存到檔案

工具清單 = [工具A, 工具B]

=== Agent 決策迴圈 ===

函式 Agent執行(任務描述):

重複執行 直到任務完成:

```
給 LLM 的輸入 = {
    系統指令: "你是一個能操作檔案的 AI 助理",
    任務: 任務描述,
    可用工具: 工具清單的名稱與說明,
    過去執行結果: 執行歷史
}
```

LLM 決策 = 呼叫 LLM(給 LLM 的輸入)

如果 LLM 決定使用某個工具:

執行該工具, 取得結果

將結果記錄到執行歷史

否則 (LLM 認為任務完成):

回傳最終答案

練習題 / 作業

課堂思考練習

練習 1: 找出 AI 可以幫你的事

列出你目前工作或日常生活中, 3 件可以讓 AI 幫你做的事:

#	任務說明	目前需要多少時間	AI 能省下什麼?
1			
2			
3			

練習 2: AI 適用性判斷

針對以下情境, 判斷是否適合讓 AI 自動完成, 並說明理由:

情境	適合?	理由
撰寫一封客戶感謝信		
計算財務報表的精確數字		
整理 200 份問卷回饋的主題		
確認法律合約中的特定條款責任		

課後作業

在下堂課之前，請完成以下任務：

1. **安裝開發環境**：確保 VS Code 與 GitHub Copilot 已正常運作
2. **AI 任務實驗**：選一件你日常工作中的小任務，用 ChatGPT 或 Claude 幫你完成
3. **記錄觀察**，下堂課分享（2-3 分鐘）：
 - 你想完成什麼任務？
 - 你如何描述需求給 AI？（你輸入了什麼 Prompt？）
 - AI 給出了什麼回應？
 - 結果是否符合你的期望？哪裡還不夠好？

延伸閱讀

AI 基礎概念：

- OpenAI 官方文件 — <https://platform.openai.com/docs>
- GitHub Copilot 入門指南 — <https://docs.github.com/en/copilot>

AI Agent 深入了解：

- LangChain 官方文件 — <https://docs.langchain.com>（主流 Agent 開發框架）
- 《Building LLM-Powered Applications》— 實用的 LLM 應用開發指南

資訊安全：

- OWASP LLM Top 10 — LLM 應用十大安全風險清單（可搜尋「OWASP LLM Top 10」取得最新版）
- NIST AI 風險管理框架 — 企業導入 AI 的風險評估指引

7/28 | 第02堂 | AI Agent 邏輯建構與自動化引擎配置

學習目標

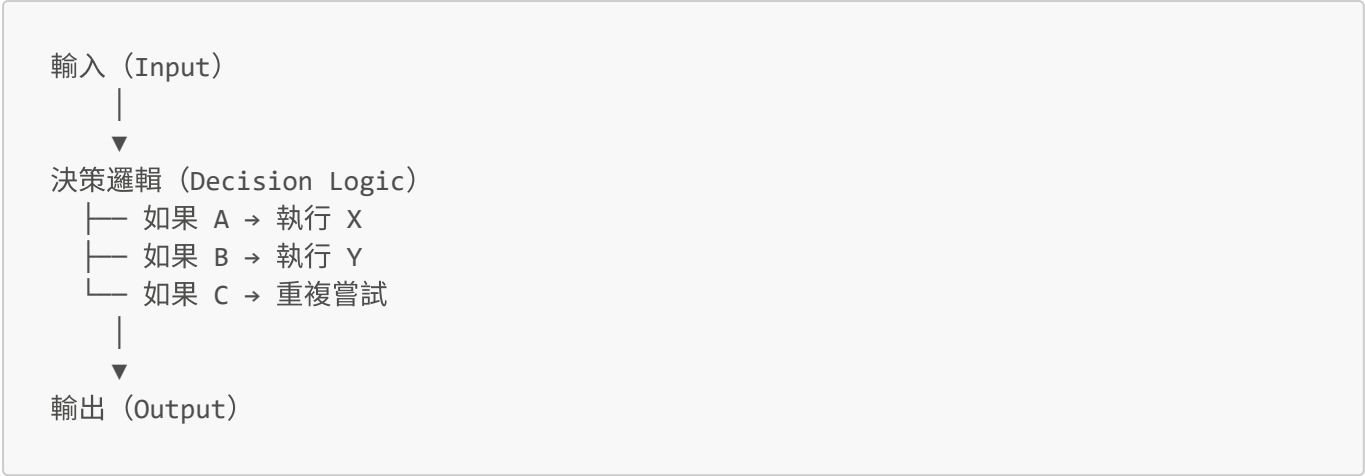
完成本堂課後，學員將能夠：

- 理解 AI Agent 流程自動化工具的核心架構與底層邏輯
- 掌握「條件判斷」與「迴圈」這兩個自動化流程的基本控制元素
- 為自動化流程加入錯誤偵測與容錯補償機制
- 使用 GitHub Copilot 撰寫一個能與 LLM 對話的 Python 程式
- 讓 Agent 具備「工具調用」能力，理解 Agent 與普通對話的本質差異

核心概念說明

1. 自動化引擎的核心架構

一個自動化流程（不論是人工執行的 SOP，還是 AI Agent），都可以拆解為三個核心元素：



對 AI Agent 而言，**決策邏輯**是由 LLM 動態生成的，而不是人工寫死的規則。

AI Agent 自動化流程的完整運作週期：

階段	說明	對應程式概念
接收任務	使用者或系統給出指令	函式的輸入參數
拆解步驟	LLM 規劃執行計畫	演算法設計
執行行動	呼叫工具完成子任務	函式呼叫
確認結果	判斷子任務是否成功	條件判斷
錯誤處理	若失敗則嘗試替代方案	例外處理
回報完成	輸出最終結果	回傳值

2. 控制流：條件判斷（Condition）

條件判斷是自動化流程中最基本的決策機制：「如果滿足某個條件，就做 A；否則就做 B。」

實際情境範例：

你要設計一個自動回覆郵件的 Agent：

- **如果** 郵件主旨包含「請款」→ 轉交財務處理
- **如果** 郵件主旨包含「抱怨」→ 轉交客服主管
- **否則** → 自動生成標準回覆

條件判斷的虛擬碼結構：

```
如果 條件A 為真：
    執行動作A
否則如果 條件B 為真：
    執行動作B
否則：
    執行預設動作
```

在 Agent 中的應用：

```
如果 LLM 的決策 == "呼叫工具":
    執行指定工具
否則如果 LLM 的決策 == "任務完成":
    回傳最終結果
否則如果 LLM 的決策 == "需要更多資訊":
    詢問使用者補充資訊
```

3. 控制流：迴圈（Loop）

迴圈讓程式能夠重複執行某段邏輯，直到滿足特定條件為止。這在 Agent 中非常關鍵——Agent 必須持續「思考 → 行動 → 觀察結果 → 再思考」，直到任務完成。

兩種常見的迴圈類型：

類型	說明	應用場景
計數迴圈	執行固定次數後停止	批次處理 100 筆資料
條件迴圈	滿足條件才停止	Agent 持續執行直到任務完成

Agent 執行迴圈（條件迴圈）：

```
重複執行 直到 任務完成：
    思考當前狀態
    決定下一個行動
    執行行動
    觀察結果
    更新對任務的理解
```

批次處理迴圈（計數迴圈）：

```
對 每一筆資料 在 資料清單 中：
    處理這筆資料
    儲存處理結果
```

注意： 條件迴圈需要確保終止條件一定能被達到，否則程式會陷入「無限迴圈」，永遠無法停止。Agent 設計中通常會設定「最大嘗試次數」作為安全機制。

4. 錯誤處理與容錯補償機制

自動化系統在執行過程中，錯誤幾乎是必然發生的：網路連線中斷、API 回應逾時、資料格式不符.....等等。
好的自動化系統必須能夠：

- 1. **偵測錯誤**：知道哪裡出了問題
- 2. **分類錯誤**：是可以重試的暫時性錯誤，還是必須停止的嚴重錯誤？
- 3. **補償行動**：執行替代方案或通知人工介入

常見的容錯策略：

策略	說明	適用情境
重試 (Retry)	自動重新嘗試，最多 N 次	API 暫時逾時、網路不穩定
等待後重試 (Exponential Backoff)	每次重試間隔越來越長	API 呼叫頻率限制
替代方案 (Fallback)	主要方案失敗時，使用備用方案	主要 LLM 故障時換用備用模型
人工介入通知	發送警告，請人工處理	無法自動修復的嚴重錯誤

5. 工具調用 (Tool Use)：讓 Agent 能「做事」

單純的 LLM 只能「說話」，而 Agent 的強大之處在於能夠「做事」——這是透過**工具調用 (Tool Use)**實現的。

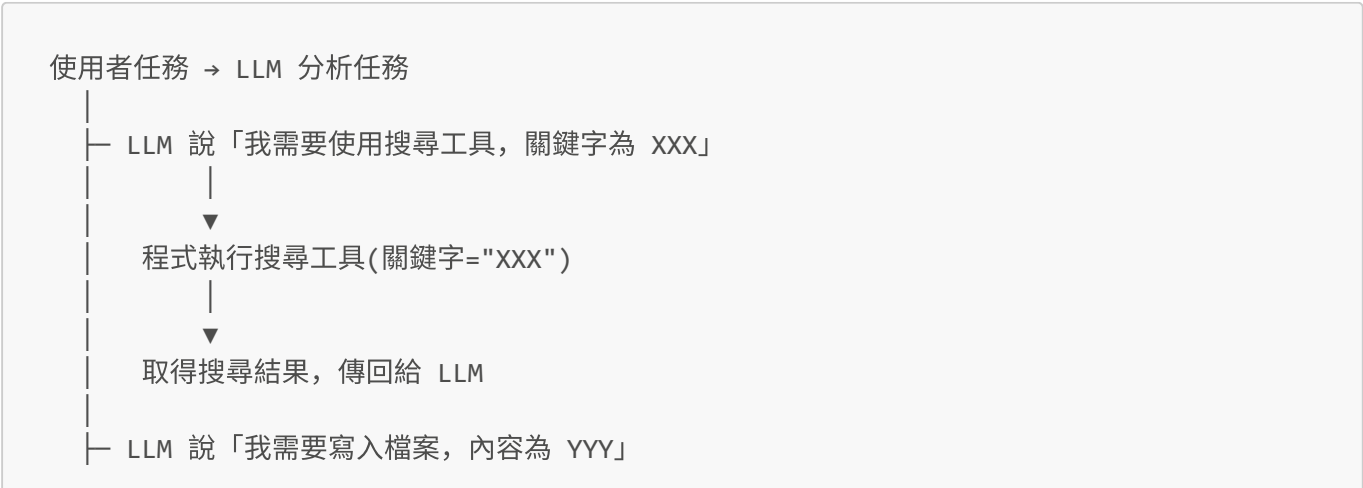
工具的概念：

工具是一個有明確輸入與輸出的函式。Agent (LLM) 在決策時，會選擇呼叫哪個工具、傳入什麼參數，然後根據工具的回傳結果決定下一步。

常見工具類型：

工具類型	功能	範例
搜尋工具	搜尋網路或資料庫	Google Search API
檔案工具	讀取/寫入本機檔案	讀取 CSV、寫入 TXT
計算工具	執行數學運算	Python 計算機
通訊工具	發送郵件、訊息	Outlook、Line API
瀏覽器工具	操控網頁	Playwright、Selenium

工具調用的完整流程：



▼
程式執行寫入檔案工具(內容="YYY")

└ LLM 說「任務完成」→ 回傳最終答案

實作步驟（逐步教學）

實作一：用 GitHub Copilot 寫一個基本的 LLM 對話程式

步驟 1：建立 Python 環境

開啟 VS Code 終端機，依序輸入以下指令：

```
安裝 openai 套件  
pip install openai  
  
確認安裝成功  
pip show openai
```

步驟 2：設定 API Key 環境變數

API Key 是存取 LLM 服務的身份憑證，**絕對不能直接寫在程式碼裡**（避免不小心上傳到 GitHub 等公開平台）。

1. 在 VS Code 終端機設定環境變數（Windows）：

```
set OPENAI_API_KEY=你的APIKey
```

2. 確認設定成功：

```
echo %OPENAI_API_KEY%
```

步驟 3：向 GitHub Copilot 描述需求

在 Copilot Chat 輸入：

```
請幫我用 Python 寫一個與 OpenAI GPT 對話的程式：  
1. 從環境變數讀取 API Key，不要寫死在程式裡  
2. 支援連續對話（保存對話歷史）  
3. 使用者輸入 "exit" 就結束程式  
4. 程式要有基本的錯誤處理（例如 API 連線失敗要顯示友善的錯誤訊息）
```

步驟 4：檢視並理解生成的程式碼

重點觀察：

- `messages` 變數如何保存對話歷史？
- `try / except` 是如何處理錯誤的？
- `os.environ` 是如何讀取環境變數的？

實作二：加入檔案工具，讓程式升級為 Agent

步驟 1：向 Copilot 請求擴充功能

請在剛才的程式基礎上，加入以下功能：

1. 定義兩個工具函式：`read_file(path)` 和 `write_file(path, content)`
2. 讓使用者可以在對話中要求 AI 讀取或寫入指定的文字檔案
3. AI 決定要用哪個工具時，由程式呼叫對應的函式，並把結果傳回給 AI

步驟 2：測試 Agent 的工具調用

用以下指令測試你的 Agent：

1. 「請幫我在桌面建立一個 `test.txt`，內容是今天的工作計畫。」
2. 「請幫我讀取桌面的 `test.txt`，告訴我裡面有什麼內容。」

步驟 3：觀察與思考

- Agent 如何決定要用哪個工具？
- 如果你給 Agent 的指令很模糊，它會怎麼處理？
- 如果工具執行失敗（例如檔案不存在），會發生什麼？

程式碼範例（虛擬碼）

含錯誤處理的 LLM 對話程式

```
# === 設定 ===
API_KEY = 從環境變數讀取("OPENAI_API_KEY")
最大重試次數 = 3
對話歷史 = [
    {"角色": "系統", "內容": "你是一個專業的工作助理，使用繁體中文回應"}
]

# === 含重試的 API 呼叫函式 ===
函式 呼叫LLM(對話歷史):
    對 嘗試次數 從 1 到 最大重試次數:
        嘗試:
            結果 = 呼叫 OpenAI API(對話歷史)
            回傳 結果
        如果發生 連線錯誤:
            顯示 f"連線失敗，第 {嘗試次數} 次重試..."
            等待 2 秒
```

```

    如果發生 其他錯誤:
        顯示 "發生未知錯誤, 停止重試"
        拋出例外

```

```

    顯示 "已達最大重試次數, 請檢查網路連線"
    結束程式

```

```

# === 主對話迴圈 ===

```

```

重複執行:

```

```

    使用者輸入 = 從終端機讀取()

```

```

    如果 使用者輸入 == "exit":
        結束程式

```

```

    將使用者輸入加入對話歷史
    AI 回應 = 呼叫LLM(對話歷史)
    將 AI 回應加入對話歷史
    顯示 AI 回應

```

帶工具調用的 Agent 虛擬碼

```

# === 定義工具 ===

```

```

函式 讀取檔案(檔案路徑):

```

```

    嘗試:

```

```

        內容 = 開啟並讀取(檔案路徑)

```

```

        回傳 內容

```

```

    如果發生錯誤:

```

```

        回傳 "錯誤: 檔案不存在或無法讀取"

```

```

函式 寫入檔案(檔案路徑, 文字內容):

```

```

    嘗試:

```

```

        開啟檔案(檔案路徑) 並寫入 文字內容

```

```

        回傳 "成功寫入檔案"

```

```

    如果發生錯誤:

```

```

        回傳 "錯誤: 無法寫入檔案"

```

```

工具清單 = {

```

```

    "讀取檔案": 讀取檔案,

```

```

    "寫入檔案": 寫入檔案

```

```

}

```

```

# === Agent 主迴圈 ===

```

```

函式 Agent執行(使用者指令):

```

```

    對話歷史 = [系統提示 + 工具說明 + 使用者指令]

```

```

    重複執行 最多 10 次:

```

```

        LLM回應 = 呼叫LLM(對話歷史)

```

```

        如果 LLM回應.類型 == "呼叫工具":

```

```

            工具名 = LLM回應.工具名稱

```

```

            參數 = LLM回應.工具參數

```

```
工具結果 = 工具清單[工具名](參數)
將工具結果加入對話歷史
```

```
否則如果 LLM回應.類型 == "最終回答":
    回傳 LLM回應.內容
```

```
回傳 "已達最大步驟數，任務未完成"
```

練習題 / 作業

課堂練習

練習 1：流程設計

設計一個「自動分類郵件」的 Agent 流程，用文字或流程圖描述：

- 輸入是什麼？（郵件主旨 + 內文）
- 需要哪些條件判斷？
- 可能發生哪些錯誤？要怎麼處理？
- 最終輸出是什麼？

練習 2：判斷迴圈類型

針對以下場景，判斷應使用「計數迴圈」還是「條件迴圈」，並說明理由：

場景	迴圈類型	理由
處理一個資料夾內的 50 個 Excel 檔案		
反覆呼叫 API 直到取得有效回應		
Agent 持續規劃執行直到任務完成		
每天定時發送報表，執行 30 天		

課後作業

1. **改良你的 Agent：**在課堂實作的程式基礎上，新增第三個工具「列出資料夾內的所有檔案」，讓 Agent 能夠知道某個資料夾有哪些檔案。
2. **錯誤情境測試：**故意製造錯誤情境（例如讀取一個不存在的檔案），觀察你的程式如何回應，並思考如何改進錯誤訊息讓它更清楚。

延伸閱讀

Python 基礎：

- Python 官方教學（中文） — <https://docs.python.org/zh-tw/3/tutorial/>
- W3Schools Python 教學 — <https://www.w3schools.com/python/>

AI Agent 架構深入：

- OpenAI Function Calling 文件 — <https://platform.openai.com/docs/guides/function-calling>
- ReAct：Agent 思考與行動框架論文（可搜尋「ReAct: Synergizing Reasoning and Acting in Language Models」）

錯誤處理最佳實務：

- Python 例外處理（Exception Handling）官方文件 — <https://docs.python.org/3/tutorial/errors.html>

8/04 | 第03堂 | 巨量數據調度工程與非結構化資料轉換

學習目標

完成本堂課後，學員將能夠：

- 說明 AI Agent 處理大量資料的典型架構與流程
- 區別「結構化資料」與「非結構化資料」的差異
- 使用程式對試算表（Excel / CSV）進行讀取、篩選、排序與寫入
- 建立批次處理流程，自動化彙整來自多份活頁簿的異質資料

核心概念說明

1. 為什麼資料處理是 Agent 的核心能力？

在企業環境中，大量的工作時間花費在「整理資料」上：

- 把多份 Excel 報表合併成一份
- 從上百筆記錄中找出符合條件的資料
- 將不同格式的資料統一成標準格式

這些工作**重複性高、規則性強**，是 AI Agent 最能發揮價值的場景。

一個典型的企業痛點：

每個月月底，業務助理要花一整天，手動把 20 個業務員各自回傳的 Excel 報價單，複製貼上到一份匯總表裡，然後再計算每個業務的總業績。

如果用 Agent 處理，整個流程可以在幾分鐘內完成。

2. 結構化資料 vs. 非結構化資料

	結構化資料	非結構化資料
定義	有固定格式，可直接用程式讀取	沒有固定格式，需要解析才能使用
範例	Excel 表格、CSV 檔、資料庫	PDF 報告、電子郵件、掃描圖片
處理難度	相對容易	需借助 AI 或 OCR 技術

結構化資料	非結構化資料
典型工作	篩選、排序、計算
	資訊提取、摘要、分類

從非結構化轉為結構化的典型流程：



3. Excel / CSV 自動化處理

在 Python 中，處理試算表最常用的工具是：

工具	適用情境	特點
openpyxl	讀寫 <code>.xlsx</code> 格式的 Excel	可保留原有格式、樣式
pandas	大量資料的分析與計算	功能強大，適合數據分析
csv 模組	讀寫 <code>.csv</code> 純文字表格	輕量，適合簡單場景

試算表操作的基本動作：

動作	說明
讀取 (Read)	從檔案載入資料到程式
篩選 (Filter)	只取出符合條件的列
排序 (Sort)	依照某欄位大小排列順序
計算 (Calculate)	加總、平均、計數等
寫入 (Write)	把處理結果存回檔案

4. 跨活頁簿資料整合（批次處理）

****批次處理 (Batch Processing) ****是指一次性處理大量同類型任務，而不是一筆一筆手動處理。

跨活頁簿整合的典型場景：

資料夾結構：

/業務報表/

├─ 北區業務_王大明.xlsx

├─ 中區業務_李小華.xlsx

├─ 南區業務_張美玲.xlsx

└─ ... (共 20 個檔案)

目標：

自動讀取所有檔案 → 彙整到一份 匯總表.xlsx → 計算各業務業績合計

批次處理的核心邏輯：

步驟1：找到所有需要處理的檔案

列出資料夾內所有 .xlsx 檔案 → 得到一個檔案清單

步驟2：逐一處理每份檔案（迴圈）

對 每個檔案 在 檔案清單 中：

讀取該檔案的資料

提取需要的欄位

將資料加入「總彙整清單」

步驟3：整理與計算

對 總彙整清單 進行排序、計算、格式化

步驟4：輸出結果

將整理後的資料寫入新的 Excel 檔案

5. AI 在資料處理中的角色

傳統的資料處理需要明確的規則（「如果欄位 A 等於 X，就...」）。但現實世界的資料往往**格式不統一**，這時 AI 就能發揮作用：

AI 的三個關鍵角色：

角色	說明	範例
格式識別	理解不同格式的資料代表什麼	「NT\$12,000」「12000元」都代表 12000 元
缺失值補充	根據上下文推斷缺失的資訊	從「台北辦公室」推斷地區為「北區」
異常偵測	找出資料中不合理的值	業績填寫為負數 → 可能是錯誤

實作步驟（逐步教學）

實作一：讀取並篩選 Excel 資料

步驟 1：安裝必要套件

```
安裝 openpyxl 套件：  
pip install openpyxl  
  
安裝 pandas 套件：  
pip install pandas openpyxl
```

步驟 2：準備測試資料

建立一個 `sample_data.xlsx`，包含以下欄位：

業務員	地區	銷售金額	日期
王大明	北區	50000	2025/07
李小華	中區	32000	2025/07
張美玲	南區	68000	2025/07
...	...	...	...

步驟 3：向 GitHub Copilot 描述需求

```
請幫我用 Python 和 pandas 寫一個程式：  
1. 讀取 sample_data.xlsx 的資料  
2. 篩選出「銷售金額」超過 40000 的記錄  
3. 依照「銷售金額」由高到低排序  
4. 計算所有業務員的銷售金額總計  
5. 把結果輸出到新的 filtered_result.xlsx 檔案
```

實作二：批次彙整多份活頁簿

步驟 1：準備多份測試 Excel 檔案

在 `報表資料/` 資料夾中建立多個 `xlsx` 檔案（可手動建立 3-5 個模擬檔案）。

步驟 2：向 GitHub Copilot 描述需求

```
請幫我用 Python 寫一個批次處理程式：  
1. 掃描「報表資料」資料夾內所有的 .xlsx 檔案  
2. 讀取每個檔案的資料（假設欄位格式相同）  
3. 把所有資料合併到一個 DataFrame  
4. 新增一個「來源檔案」欄位，記錄每筆資料來自哪個檔案  
5. 依照業務員計算各自的銷售金額加總  
6. 將匯總結果輸出到「月份彙整報表.xlsx」
```

步驟 3：加入 AI 輔助的格式識別

在程式中加入 LLM 呼叫，讓 AI 幫助你處理格式不一致的問題：

- 有些欄位名稱不同（「業務」vs「業務員」）
- 有些金額格式不同（「\$50,000」vs「50000」）

實作三：AI Agent 驅動的資料清理

步驟 1：描述整體 Agent 流程

我要建立一個資料清理 Agent：

1. 讀取一份包含格式不一致資料的 Excel 檔
 2. 對每一列資料，呼叫 LLM API 進行格式標準化
 3. 將清理後的資料寫回新的 Excel 檔案
 4. 如果某一列資料 AI 無法確定，標記為「需人工審查」
- 請幫我設計這個程式的架構

程式碼範例（虛擬碼）

讀取與篩選試算表

載入 pandas 套件

```
# 讀取 Excel 檔案
```

```
資料表 = 讀取Excel("sample_data.xlsx", 工作表="Sheet1")
```

```
# 查看資料基本資訊
```

```
顯示 資料表的前5行
```

```
顯示 資料表的欄位名稱
```

```
顯示 資料表的資料筆數
```

```
# 篩選：銷售金額超過 40000
```

```
篩選結果 = 資料表[ 資料表["銷售金額"] > 40000 ]
```

```
# 排序：由高到低
```

```
排序結果 = 篩選結果.依照("銷售金額", 由大到小=真)
```

```
# 計算加總
```

```
總計 = 篩選結果["銷售金額"].加總()
```

```
顯示 f"篩選後總計: {總計}"
```

```
# 輸出到新 Excel 檔案
```

```
排序結果.寫入Excel("filtered_result.xlsx", 不含索引=真)
```

```
顯示 "結果已儲存"
```

批次彙整多份活頁簿

```

載入 pandas, os, pathlib 套件

資料夾路徑 = "報表資料/"
所有資料 = [] # 用來收集所有檔案的資料

# 掃描資料夾內所有 xlsx 檔案
對 每個檔案 在 列出資料夾(資料夾路徑, 篩選="*.xlsx") 中:
    顯示 f"正在處理: {檔案名稱}..."

    # 讀取當前檔案
    當前資料 = 讀取Excel(檔案路徑)

    # 加入「來源檔案」欄位
    當前資料["來源檔案"] = 檔案名稱

    # 加入總清單
    所有資料.加入(當前資料)

# 合併所有資料
彙整資料 = 合併所有(所有資料)

# 依業務員計算加總
業績彙總 = 彙整資料.依照("業務員").計算加總("銷售金額")

# 輸出
業績彙總.寫入Excel("月份彙整報表.xlsx")
顯示 f"完成! 共處理 {len(所有資料)} 個檔案"

```

AI 輔助的格式標準化

```

函式 標準化欄位(原始值, 欄位類型):
    提示詞 = f"""
    我有一筆資料: "{原始值}"
    這是「{欄位類型}」欄位的值。
    請將它轉換為標準格式:
    - 如果是金額: 只回傳數字 (不含符號與逗號)
    - 如果是日期: 轉換為 YYYY-MM-DD 格式
    - 如果無法判斷: 只回傳「UNKNOWN」
    只回傳結果, 不要說明。
    """
    回傳 呼叫LLM(提示詞)

# 對每一列資料進行標準化
對 每一列資料 在 資料表 中:
    資料["銷售金額"] = 標準化欄位(資料["銷售金額"], "金額")
    資料["日期"] = 標準化欄位(資料["日期"], "日期")

    如果 任何欄位 == "UNKNOWN":
        資料["備註"] = "需人工審查"

```

練習題 / 作業

課堂練習

練習 1：設計資料處理流程

你的公司有以下情境，請設計一個 Agent 的處理流程（可用文字或流程圖）：

每週一，各部門主管會以電子郵件回傳一份本週銷售紀錄的 Excel 檔案（格式略有不同）。需要自動彙整所有部門的銷售資料，計算各部門總業績，並生成一份比較報表。

請說明：

- 需要哪些步驟？
- 每個步驟的輸入和輸出是什麼？
- 哪些地方可能出現問題？

練習 2：資料識別

以下資料代表同樣的含義，但格式不同。請判斷 AI 需要做哪些處理才能統一它們：

原始資料	應標準化為
NT\$12,500	
1萬2千5百元	
12500.00	
\$12,500 USD	

課後作業

1. **動手實作：**在你的電腦上建立 3 份格式相同的 Excel 檔案（可手動填寫一些測試資料），然後用 GitHub Copilot 協助你寫一個批次彙整程式。
2. **格式挑戰：**將 3 份 Excel 故意使用不同的欄位名稱（如：「姓名」vs「業務員」vs「員工」），看看你的程式能否正確合併，或是需要額外的處理。

延伸閱讀

工具文件：

- pandas 官方文件 — <https://pandas.pydata.org/docs/>
- openpyxl 官方文件 — <https://openpyxl.readthedocs.io/>

資料處理概念：

- 《Python for Data Analysis》by Wes McKinney — pandas 作者撰寫的資料分析聖經
- ETL（Extract, Transform, Load）概念介紹 — 可搜尋「ETL pipeline 教學」

AI + 資料處理：

- LangChain CSV Agent 教學 — LangChain 官方文件的 CSV 處理章節

8/11 | 第04堂 | 跨平台 Web 數據擷取與異質系統整合

學習目標

完成本堂課後，學員將能夠：

- 說明 Web 自動化與 Web 爬蟲的差異及各自的應用場景
- 使用瀏覽器自動化工具模擬使用者操作（登入、填表、擷取資料）
- 了解 Docker 的基本概念，並學會安裝與啟動容器
- 認識 NemoClaw（NVIDIA 對 OpenClaw 的延伸產品），並完成基礎環境配置

核心概念說明

1. 為什麼需要 Web 自動化？

許多企業的重要資料仍然存放在各種網路平台上，例如：

- 政府開放資料平台
- 電商平台上的競品價格
- 產業研究報告網站
- 公司內部的 ERP / CRM 系統

這些資料通常無法透過 API 直接取得，只能透過「模擬人類操作瀏覽器」的方式來擷取。

2. Web 爬蟲 vs. Web 自動化

這兩個概念常被混用，但技術上有明確的差異：

	Web 爬蟲（Scraping）	Web 自動化（Automation）
核心目的	大量擷取網頁上的文字資料	模擬使用者在瀏覽器的操作行為
主要技術	requests + BeautifulSoup	Playwright、Selenium
能做什麼	讀取靜態網頁內容	點擊按鈕、填表、登入、處理動態頁面
適用情境	新聞抓取、靜態資料庫	需要登入的系統、動態載入的內容
限制	無法處理 JavaScript 動態內容	速度較慢，資源消耗較多

3. 瀏覽器自動化技術：Playwright

Playwright 是由 Microsoft 開發的現代瀏覽器自動化框架，能夠控制 Chrome、Firefox、Safari 等瀏覽器，執行各種使用者操作。

Playwright 能做到的事：

操作	說明
開啟網頁	導航至任意 URL
點擊元素	點擊按鈕、連結、核取方塊
填寫表單	輸入文字、選擇下拉選單
擷取資料	取得網頁上的文字、圖片、連結
截圖	對目前頁面截圖
等待元素	等到特定內容出現才繼續
處理彈出視窗	確認對話方塊、切換分頁
下載檔案	觸發下載並儲存到本機

認證流程自動化（重要概念）：

很多平台需要先登入才能存取資料。自動化登入的基本流程：

1. 開啟登入頁面
2. 找到「帳號」輸入框，填入帳號
3. 找到「密碼」輸入框，填入密碼
4. 點擊「登入」按鈕
5. 等待頁面跳轉完成
6. 確認登入成功（尋找登入後才有的元素）
7. 繼續執行後續任務

資安提醒： 帳號密碼務必從環境變數讀取，絕對不能寫死在程式碼中，更不能上傳到 GitHub。

4. 網路反爬蟲機制與道德規範

技術面的挑戰：

許多網站會設置防護機制來阻擋自動化程式：

防護機制	說明
CAPTCHA	要求驗證使用者是人類
速率限制	太頻繁的請求會被封鎖
User-Agent 檢查	識別並封鎖非瀏覽器的請求
IP 封鎖	短時間大量請求會被封鎖 IP

道德與法律規範：

- 使用前請閱讀網站的**服務條款（Terms of Service）**，確認是否允許自動化存取
- 避免對目標網站造成不必要的伺服器負載（請求間隔應適當）
- 不應用於爬取受版權保護的內容或個人隱私資料

- 若網站提供 API，應優先使用 API 而非爬蟲

5. Docker 基礎概念

Docker 是一種「容器化」技術，可以把程式及其所有相依套件**打包在一個獨立、可攜帶的容器中**，確保在任何電腦上都能以完全相同的方式執行。

一個簡單的比喻：

想像 Docker 容器就像是一個**預先配置好環境的虛擬電腦**：

- 所有需要的軟體都已預先安裝
- 不會受到你電腦現有環境的干擾
- 在任何機器上執行結果都一樣

Docker 的核心概念：

概念	說明	生活比喻
Image（映像檔）	包含程式與環境的模板	食譜
Container（容器）	根據 Image 運行的實際程序	依照食譜做出來的料理
Registry	存放 Image 的倉庫	食譜書
Volume	容器與主機間的資料共享	共用材料箱

常用 Docker 指令（示意）：

```
# 下載 Image
docker pull [image名稱]

# 啟動容器
docker run [選項] [image名稱]

# 查看執行中的容器
docker ps

# 停止容器
docker stop [容器ID]
```

6. NemoClaw 簡介

NemoClaw 是 NVIDIA 基於 OpenClaw 框架延伸開發的 AI Agent 平台，針對 **GPU 加速運算**做了深度優化，特別適合：

- 需要大量本地 AI 推理的場景
- 處理敏感資料（無需將資料送至外部 API）
- 高效能的文件理解與多模態任務

NemoClaw 的主要優勢（相較於純 OpenClaw）：

特點	說明
本地 LLM 運行	不依賴外部 API，資料不離開本機
NVIDIA GPU 加速	大幅提升推理速度
企業級安全	適合處理機密或個資資料
多模態支援	可處理文字、圖片、PDF 等多種格式

NemoClaw 的安裝方式（透過 Docker）：

NemoClaw 提供官方 Docker 映像檔，透過 Docker 可以快速完成安裝，不需要複雜的環境配置，這也是本堂課選擇先介紹 Docker 的原因。

實作步驟（逐步教學）

實作一：安裝 Playwright 並執行第一個自動化腳本

步驟 1：安裝 Playwright

```
安裝 playwright 套件：  
    pip install playwright  
  
下載瀏覽器驅動程式：  
    playwright install chromium
```

步驟 2：向 GitHub Copilot 描述你的第一個自動化需求

```
請幫我用 Python 和 Playwright 寫一個程式：  
1. 開啟 Chrome 瀏覽器（有視覺畫面，不要用無頭模式）  
2. 前往 https://www.google.com  
3. 在搜尋框輸入「AI Agent 教學」  
4. 點擊搜尋按鈕  
5. 等待搜尋結果載入後，擷取前5個搜尋結果的「標題」和「網址」  
6. 把結果列印在終端機上
```

步驟 3：觀察瀏覽器的操作過程

執行程式後，你會看到一個 Chrome 瀏覽器視窗自動開啟並完成搜尋，理解整個操作流程。

實作二：模擬登入某平台並擷取資料

本實作以一個測試用的範例網站進行演示（不針對任何特定商業平台）。

步驟 1：向 Copilot 描述需求

請幫我用 Playwright 寫一個自動登入程式：

1. 從環境變數讀取帳號 (USERNAME) 和密碼 (PASSWORD)
 2. 開啟目標網站的登入頁面
 3. 找到帳號和密碼輸入框，填入認證資訊
 4. 點擊登入按鈕
 5. 等待跳轉，確認登入成功 (檢查頁面標題或特定元素是否存在)
 6. 導航到資料頁面，擷取表格資料
 7. 把資料存成 CSV 檔案
- 請加入適當的等待機制和錯誤處理

步驟 2：理解「等待機制」的重要性

網頁通常需要一些時間才能載入完成，自動化程式必須「等待」元素出現才能操作：

等待類型	說明	適用場景
固定等待	強制等待固定秒數	簡單但效率低，不建議
等待元素出現	等到特定元素出現才繼續	最常用、最可靠
等待網路靜止	等到網頁不再有網路請求	初始頁面載入

實作三：安裝 Docker 並部署 NemoClaw

步驟 1：安裝 Docker Desktop

1. 前往 <https://www.docker.com/products/docker-desktop/> 下載 Docker Desktop
2. 完成安裝後重新開機
3. 開啟 Docker Desktop，確認右下角顯示「Docker is running」

步驟 2：確認 Docker 正常運作

開啟終端機，輸入：

```
docker --version
docker run hello-world
```

若看到歡迎訊息，表示 Docker 安裝成功。

步驟 3：依照 NemoClaw 官方文件拉取映像檔

```
# 下載 NemoClaw 映像檔 (依官方文件的實際指令執行)
docker pull nvcr.io/nvidia/nemo:latest

# 啟動 NemoClaw 容器
docker run -d \
  --name nemo \
```

```
-p 8080:8080 \  
nvcr.io/nvidia/nemoclave:latest
```

步驟 4：連接 NemoClaw 介面並設定 workflow

1. 開啟瀏覽器，前往 <http://localhost:8080>
2. 完成初始化設定
3. 建立第一個 NemoClaw workflow，連接剛才的 Web 自動化腳本作為工具節點

程式碼範例（虛擬碼）

Playwright 基礎自動化

```
載入 playwright 套件  
  
# 啟動瀏覽器  
啟動 playwright:  
    瀏覽器 = 開啟Chromium(有視覺畫面=真)  
    頁面 = 開新分頁()  
  
# 前往目標網址  
頁面.前往("https://目標網站.com")  
  
# 等待特定元素出現後操作  
頁面.等待元素("#帳號輸入框")  
頁面.填入("#帳號輸入框", 帳號)  
頁面.填入("#密碼輸入框", 密碼)  
頁面.點擊("#登入按鈕")  
  
# 等待登入完成  
頁面.等待URL改變()  
  
# 擷取資料  
資料表格 = 頁面.取得所有元素("table tr")  
對 每一列 在 資料表格 中:  
    擷取每個欄位的文字  
    加入資料清單  
  
# 儲存結果  
將資料清單寫入 CSV 檔案  
  
# 關閉瀏覽器  
瀏覽器.關閉()
```

含動態表單處理的進階腳本

```
函式 填寫搜尋條件(頁面, 搜尋參數):  
    # 選擇下拉選單  
    頁面.選擇("#地區選單", 搜尋參數.地區)  
  
    # 填入日期範圍  
    頁面.填入("#開始日期", 搜尋參數.開始日期)  
    頁面.填入("#結束日期", 搜尋參數.結束日期)  
  
    # 點擊搜尋  
    頁面.點擊("#搜尋按鈕")  
  
    # 等待結果載入 (等待載入動畫消失)  
    頁面.等待元素(".載入動畫", 狀態=消失)  
    頁面.等待元素(".搜尋結果", 狀態=出現)  
  
函式 擷取分頁資料(頁面):  
    所有資料 = []  
  
    # 處理多頁資料  
    重複執行:  
        當前頁資料 = 擷取表格資料(頁面)  
        所有資料.加入(當前頁資料)  
  
        下一頁按鈕 = 頁面.尋找元素(".下一頁")  
        如果 下一頁按鈕 不存在 或 下一頁按鈕 被禁用:  
            跳出迴圈  
        下一頁按鈕.點擊()  
        頁面.等待載入完成()  
  
    回傳 所有資料
```

練習題 / 作業

課堂練習

練習 1: 流程設計

設計一個「自動收集競品資訊」的 Web 自動化 Agent，描述以下內容：

- 目標平台是什麼類型的網站（電商、新聞、政府資料等）？
- 需要哪些操作步驟（不用寫程式碼，用文字描述）？
- 擷取到的資料要如何儲存和使用？

練習 2: 識別等待時機

針對以下操作，判斷應該等待什麼條件才能繼續下一步：

操作	應等待的條件
點擊「搜尋」按鈕後	
點擊「下一頁」按鈕後	
點擊「下載」按鈕後	
填入搜尋關鍵字後	

課後作業

1. **Playwright 實作**：使用 Playwright 自動化擷取一個公開資料網站（例如政府開放資料平台）的特定資料，並儲存為 CSV 或 Excel 格式。
2. **Docker 熟悉**：在自己的電腦上成功安裝 Docker Desktop，並執行官方的 `hello-world` 容器，截圖留存。

延伸閱讀

Web 自動化工具：

- Playwright 官方文件 — <https://playwright.dev/python/>
- Playwright 選擇器指南 — 如何正確定位網頁元素

Docker 學習資源：

- Docker 官方入門教學 — <https://docs.docker.com/get-started/>
- 《Docker 深入淺出》— 友善的 Docker 入門書籍

NVIDIA NemoClaw：

- NVIDIA NIM 官方文件 — <https://docs.nvidia.com/nim/>（NemoClaw 的相關技術基礎）

法律與道德：

- 《網路爬蟲的法律灰色地帶》— 建議使用前進行法律諮詢
- Robots.txt 規範說明 — 了解 robots.txt 對自動化爬蟲的指引

8/18 | 第05堂 | 智慧通訊協作：電子郵件與 Office 流程自動化

學習目標

完成本堂課後，學員將能夠：

- 建立基於 Outlook 的智慧郵件自動化流程
- 使用 AI 對郵件進行語意分析，識別意圖與優先順序
- 自動化下載與解析郵件附件
- 根據郵件內容觸發對應的回覆邏輯與任務分派
- 整合郵件與行事曆，實現會議排程自動化

核心概念說明

1. 為什麼郵件自動化是企業的重要場景？

電子郵件是企業最主要的通訊工具之一，但也是效率瓶頸的來源：

- 平均每位知識工作者每天花費 **2.5 小時**處理電子郵件
- 大量郵件是重複性的：確認收到、請款、詢問狀態
- 重要郵件容易淹沒在垃圾訊息中，造成遺漏

AI Agent 能夠 24 小時不間斷地監控、分類、回覆並分派郵件任務。

2. Outlook 自動化的技術選項

在 Windows 環境下，操控 Outlook 主要有以下幾種方式：

技術	說明	適合情境
win32com (pywin32)	透過 Windows COM 介面控制 Outlook 應用程式	本地 Outlook 桌面版
Microsoft Graph API	透過 Microsoft 官方 API 存取郵件、行事曆	雲端 M365 環境
exchangelib	Python 套件，連接 Exchange Server	企業 Exchange 環境
IMAP / SMTP	標準郵件協定，跨平台	非 Outlook 的郵件服務

本堂課主要以 **win32com** 示範本地 Outlook 操作，以及介紹 **Microsoft Graph API** 的概念作為現代化替代方案。

3. 郵件的語意分析

語意分析（Semantic Analysis）是指理解文字背後**的真實意圖**，而不只是字面上的詞彙。

傳統規則方式 vs. AI 語意分析：

	傳統關鍵字規則	AI 語意分析
識別「催款」郵件	只能識別包含「請款」「付款」等關鍵字	能理解「我們上個月的發票還沒收到款項」也是催款意圖
識別「投訴」郵件	只能識別「抱怨」「不滿」等詞	能識別語氣委婉但明確表達不滿的郵件
多語言支援	需要為每種語言維護不同的規則	同樣的 Prompt 即可處理中、英、日文郵件

郵件語意分析的典型分類任務：

分析維度	可能的分類結果
意圖分類	詢問報價、確認訂單、申請退款、投訴、一般諮詢
優先級判斷	緊急、一般、低優先

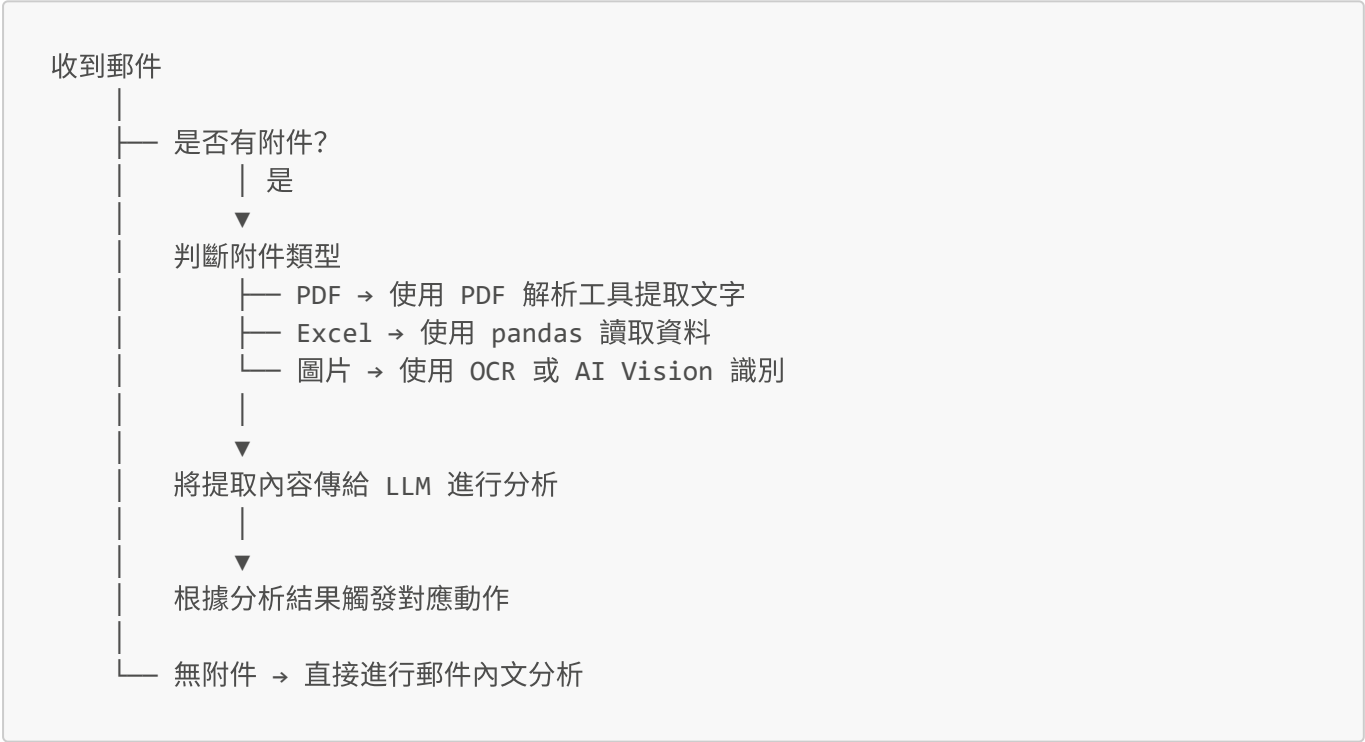
分析維度	可能的分類結果
情感分析	正面、中性、負面
需要回覆？	是、否
對應部門	業務、財務、客服、技術支援

4. 附件處理自動化

郵件附件是企業資訊交換的重要媒介，常見的附件處理需求：

- **PDF**：合約、發票、報告 → 提取文字內容
- **Excel**：資料報表 → 讀取並彙整資料
- **Word**：文件草稿 → 提取內容進行摘要
- **圖片**：掃描文件 → OCR 文字識別

附件自動化處理流程：



5. 行事曆同步自動化

常見的會議排程自動化場景：

- 客戶寄來「請問下週可以開會嗎？」→ AI 查看行事曆空檔，自動回覆可用時間
- 收到會議邀請郵件 → 自動建立行事曆事件並設定提醒
- 定期報告完成 → 自動安排下次討論會議

行事曆操作的基本元素：

元素	說明
事件標題	會議名稱
開始/結束時間	日期與時段
地點	實體地點或線上連結
與會者	邀請對象的 Email
提醒	會議前多少分鐘提醒
重複設定	每週固定時段的例行會議

實作步驟（逐步教學）

實作一：讀取 Outlook 郵件並進行分類

步驟 1：安裝必要套件

```
安裝 pywin32 (Outlook COM 介面) :  
pip install pywin32
```

```
安裝 openai (LLM API) :  
pip install openai
```

步驟 2：向 GitHub Copilot 描述需求

```
請幫我用 Python 和 pywin32 寫一個程式：  
1. 連接到本地的 Microsoft Outlook  
2. 讀取「收件匣」中最新的 20 封未讀郵件  
3. 對每封郵件提取：寄件者、主旨、收到時間、正文內容  
4. 呼叫 OpenAI API，讓 AI 判斷每封郵件的「類型」（詢問/投訴/確認/其他）  
   和「優先級」（高/中/低）  
5. 根據分類結果，印出分類統計報告
```

步驟 3：理解郵件的 COM 物件結構

Outlook 的郵件物件包含許多屬性，重點關注：

- **Subject**：郵件主旨
- **SenderEmailAddress**：寄件者地址
- **Body**：郵件內文（純文字）
- **ReceivedTime**：收到時間
- **Attachments**：附件集合
- **UnRead**：是否未讀

實作二：自動化回覆流程

步驟 1：設計回覆邏輯

請在上一個程式的基礎上，加入自動回覆功能：

1. 對於分類為「詢問報價」的郵件：
 - 讓 AI 根據郵件內容生成一封專業的初步回覆（說明需要進一步確認規格）
 - 將回覆草稿儲存到 Outlook 的「草稿箱」（不要直接發送）
2. 對於分類為「投訴」的郵件：
 - 標記為高優先級（紅色旗幟）
 - 轉寄給主管信箱（從環境變數讀取主管信箱）
 - 自動回覆客戶說明正在處理中

請注意：所有自動發送的郵件都要先存入草稿箱供人工審核，不要直接發送。

重要原則：自動化郵件系統應設計為「輔助」而非「完全取代」人工判斷。關鍵郵件務必要有人工審核環節。

實作三：附件分析與任務分派

步驟 1：加入附件自動處理

請幫我加入附件自動處理功能：

1. 偵測郵件中的附件
2. 如果是 PDF 附件：使用 pdfplumber 套件提取文字，然後讓 AI 摘要內容
3. 如果是 Excel 附件：使用 pandas 讀取，讓 AI 描述資料的基本統計
4. 將附件內容摘要加入分類結果的報告中
5. 自動將附件儲存到對應分類的資料夾（例如：「投訴資料夾」、「報價單資料夾」）

步驟 2：安裝 PDF 解析套件

```
pip install pdfplumber
```

程式碼範例（虛擬碼）

讀取並分類 Outlook 郵件

```
載入 win32com.client 套件
載入 openai 套件

# 連接 Outlook
Outlook應用 = 啟動("Outlook.Application")
名稱空間 = Outlook應用.GetNamespace("MAPI")
收件匣 = 取得資料夾("收件匣")

分類結果清單 = []
```

```

# 讀取最新 20 封未讀郵件
未讀郵件清單 = [郵件 for 郵件 in 收件匣.Items if 郵件.UnRead == 真][:20]

對 每封郵件 在 未讀郵件清單 中:
    # 提取郵件資訊
    郵件資訊 = {
        "寄件者": 郵件.SenderEmailAddress,
        "主旨": 郵件.Subject,
        "時間": 郵件.ReceivedTime,
        "內文": 郵件.Body[:1000] # 只取前 1000 字避免超過 Token 限制
    }

# 呼叫 AI 分類
提示詞 = f"""
以下是一封電子郵件:
主旨: {郵件資訊["主旨"]}
內文: {郵件資訊["內文"]}

請以 JSON 格式回傳:
{{
    "類型": "詢問/投訴/確認訂單/請款/其他",
    "優先級": "高/中/低",
    "摘要": "20字以內的摘要",
    "建議動作": "建議的處理方式"
}}
"""

AI分類結果 = 呼叫LLM(提示詞)
分類結果清單.加入({...郵件資訊, ...解析JSON(AI分類結果)})

# 輸出統計報告
顯示分類統計報告(分類結果清單)

```

自動回覆草稿生成

函式 生成回覆草稿(原始郵件, 分類結果):

```

如果 分類結果.類型 == "詢問報價":
    回覆提示詞 = f"""
    客戶寄了以下詢價郵件:
    {原始郵件.內文}

    請幫我撰寫一封專業的初步回覆, 要:
    1. 感謝客戶詢問
    2. 說明需要確認具體規格才能提供正式報價
    3. 請客戶提供: 數量、交期需求、特殊規格要求
    4. 語氣要專業友善
    5. 用繁體中文撰寫
    """
    回覆內容 = 呼叫LLM(回覆提示詞)

```

```

# 建立草稿
新郵件 = Outlook應用.CreateItem(郵件類型)
新郵件.To = 原始郵件.SenderEmailAddress
新郵件.Subject = "Re: " + 原始郵件.Subject
新郵件.Body = 回覆內容
新郵件.儲存到草稿箱() # 只存草稿，不發送

否則如果 分類結果.類型 == "投訴":
    原始郵件.設定優先級(高)
    原始郵件.轉寄給(主管信箱)
    # ... 類似邏輯

```

附件自動下載與解析

```

函式 處理附件(郵件, 儲存資料夾):
    附件摘要清單 = []

    對 每個附件 在 郵件.Attachments 中:
        附件名稱 = 附件.FileName
        本地路徑 = 儲存資料夾 + "/" + 附件名稱

        # 儲存附件到本地
        附件.SaveAsFile(本地路徑)

        # 依附件類型處理
        如果 附件名稱.結尾是(".pdf"):
            文字內容 = 使用pdfplumber提取文字(本地路徑)
            摘要 = AI摘要(文字內容)
            附件摘要清單.加入({"檔名": 附件名稱, "摘要": 摘要})

        否則如果 附件名稱.結尾是(".xlsx", ".csv"):
            資料 = 使用pandas讀取(本地路徑)
            統計描述 = AI描述資料(資料.基本統計())
            附件摘要清單.加入({"檔名": 附件名稱, "摘要": 統計描述})

    回傳 附件摘要清單

```

練習題 / 作業

課堂練習

練習 1: 郵件分類規則設計

針對你的實際工作場景，設計郵件自動分類的規則：

郵件類型	如何判斷？（關鍵語意特徵）	應採取什麼動作？

練習 2：回覆品質評估

思考以下問題：

- 哪些類型的郵件，AI 自動生成的回覆可以直接使用？
- 哪些類型的郵件，AI 的回覆一定要人工審核後才能發送？
- 如何設計一個「人機協作」的郵件回覆 workflow？

課後作業

1. **郵件分類器**：設計一個至少能分類 5 種郵件類型的 Prompt，並用你真實收到過的郵件（隱去個資）測試分類準確度。
2. **流程圖設計**：設計你理想中的「智慧郵件助理」完整工作流程，包含：觸發條件、判斷邏輯、執行動作、人工審核節點。

延伸閱讀

技術文件：

- pywin32 文件 — 搜尋「pywin32 Outlook tutorial」
- Microsoft Graph API 郵件文件 — <https://learn.microsoft.com/en-us/graph/api/resources/mail-api-overview>
- pdfplumber — <https://github.com/jsvine/pdfplumber>

郵件自動化概念：

- 《Automate the Boring Stuff with Python》第 18 章：電子郵件自動化
- Gmail API Python 快速入門 — 適用非 Outlook 環境的替代方案

8/25 | 第06堂 | 檔案系統運維與傳統系統介面整合

學習目標

完成本堂課後，學員將能夠：

- 建立複雜的檔案自動化管理系統（整理、搬移、重新命名、備份）
- 從 PDF 文件中提取非結構化文字資料並轉化為可用資訊
- 使用桌面 UI 自動化技術，讓 AI 助理能夠操控傳統桌面軟體
- 理解「遺留系統（Legacy System）整合」的概念與實務挑戰

核心概念說明

1. 為什麼「檔案管理與傳統系統整合」是實務中的重要需求？

在實際的企業環境中，有大量未被數位化或 API 化的工作：

常見的企業痛點：

情境	問題
每月收到數百份 PDF 發票	需要手動打開每份、複製金額和日期到 Excel
公司使用 20 年前的進銷存系統	系統無法匯出資料，只能靠人工抄寫
員工桌面資料夾一片混亂	找不到資料，版本管理混亂
掃描文件需要數位化	大量紙本資料未被有效利用

這些場景沒有可用的 API，只能透過「模擬人類操作」的方式自動化。

2. 檔案系統操作

Python 提供了豐富的標準函式庫來處理檔案和資料夾：

模組	主要用途
<code>os</code>	基礎的作業系統操作（路徑、目錄、環境變數）
<code>pathlib</code>	現代化的路徑處理（推薦使用）
<code>shutil</code>	檔案複製、搬移、刪除、壓縮
<code>glob</code>	依照模式搜尋檔案（支援萬用字元）
<code>watchdog</code>	監控資料夾，偵測新檔案的出現

常見的檔案自動化任務：

資料夾整理：

- 掃描指定資料夾
- 依照副檔名分類（.pdf → PDF 資料夾、.xlsx → Excel 資料夾）
- 依照日期分類（今年/去年、月份）
- 依照檔案命名規則分類

批次重新命名：

- 讀取所有符合條件的檔案
- 依照規則生成新的檔名（例如：加上日期前綴）
- 重新命名並記錄對照表

自動備份：

- 監控重要資料夾的異動
- 偵測到新增或修改時，自動複製到備份位置
- 保留指定數量的歷史版本

3. PDF 非結構化資料提取

PDF 是企業中最常見的文件格式，但它的資料不像 Excel 那樣容易讀取。

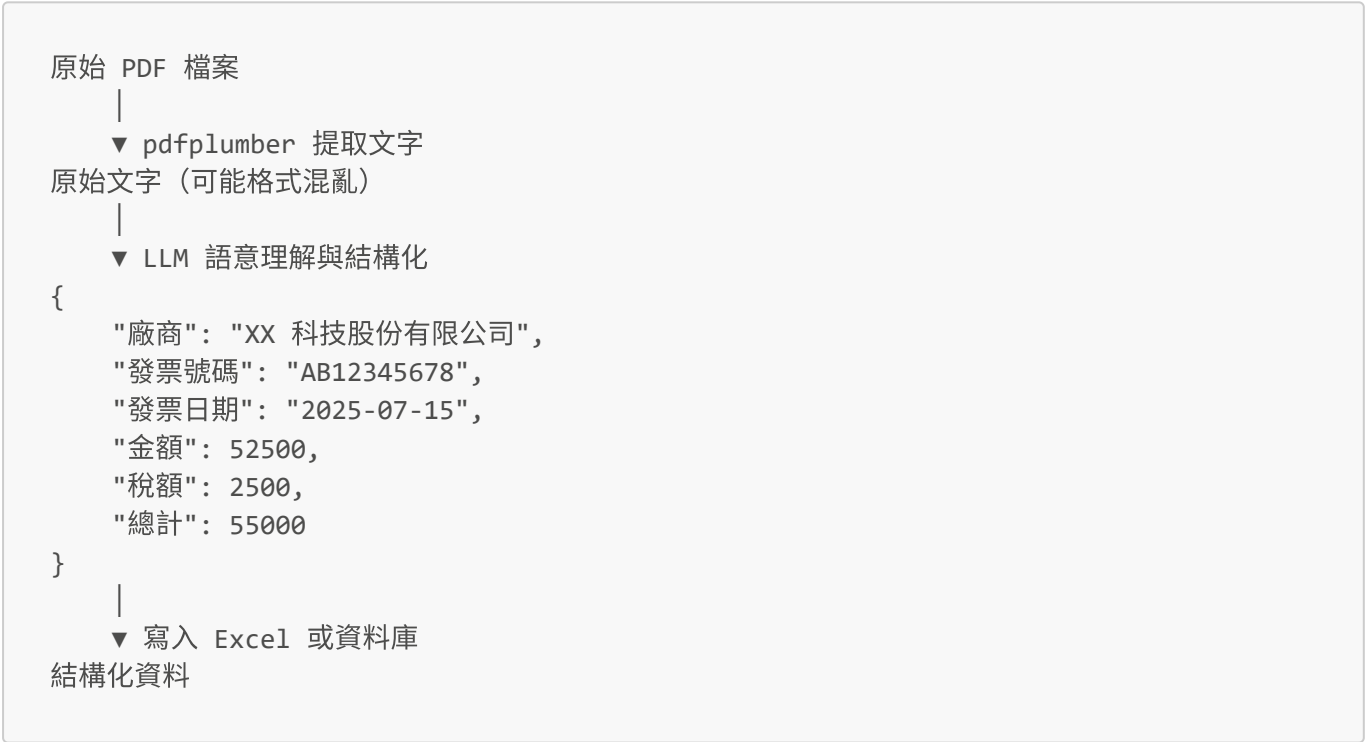
PDF 的兩種主要類型：

類型	說明	提取方式
文字型 PDF	由電腦直接生成，文字可被搜尋	pdfplumber、pdfminer
掃描型 PDF	紙本文件掃描成 PDF，本質上是圖片	OCR（光學字元識別）工具

常見的 PDF 提取工具：

工具	特色
pdfplumber	精確提取文字與表格結構，適合文字型 PDF
PyMuPDF (fitz)	速度快、功能多，適合大批量處理
Tesseract + pytesseract	開源 OCR 引擎，適合掃描型 PDF
Azure AI Document Intelligence	雲端 AI，準確率高，適合複雜版面

AI 輔助的 PDF 資料提取流程：



4. 桌面 UI 自動化 (RPA 概念)

****RPA (Robotic Process Automation，機器人流程自動化) ****是一種讓電腦程式模擬「人類操作滑鼠和鍵盤」的技術，讓傳統桌面軟體也能被自動化控制。

為什麼需要 RPA？

許多企業有使用了 10-20 年的系統：

- 老舊 ERP 系統（進銷存、財會）
- 政府平台（需要手動填寫大量欄位）
- 不提供 API 的第三方軟體

這些系統唯一的操作方式就是「人坐在電腦前點選」，RPA 讓機器人來做這件事。

Python UI 自動化工具：

工具	適用場景
pyautogui	基礎滑鼠鍵盤操控，跨平台
pygetwindow	管理視窗（最大化、切換焦點）
uiautomation	Windows 原生 UI 元素識別（推薦用於 Windows 應用程式）

UI 自動化的基本操作：

操作	說明
找到視窗	依視窗標題找到目標應用程式
點擊座標	在螢幕特定位置點擊
輸入文字	模擬鍵盤輸入
截圖比對	找到螢幕上特定的圖示或按鈕
等待元素	等待特定視覺元素出現

重要提醒： UI 自動化比 API 自動化脆弱許多——介面稍有變化（版本更新、螢幕解析度改變）就可能導致腳本失效。應加入足夠的偵錯機制，並定期維護。

5. 遺留系統整合的策略

面對無法改造的舊系統，常見的整合策略：

策略	說明	難度
UI 自動化	用機器人模擬人工操作	低
資料庫直接存取	直接讀寫舊系統的資料庫	中（需要 IT 授權）
螢幕截圖 + AI 識別	截取畫面，用 AI 理解並提取資訊	中
中介層（Middleware）	在舊系統外包一層 API	高（需要開發）

實作步驟（逐步教學）

實作一：自動整理桌面資料夾

步驟 1：安裝必要套件

```
pip install pathlib # Python 標準函式庫，通常無需安裝
pip install watchdog # 資料夾監控
```


步驟 2：向 GitHub Copilot 描述需求

請幫我寫一個 Python 程式，自動整理「下載」資料夾：

1. 掃描「下載」資料夾（路徑從環境變數讀取）
2. 依照以下規則自動分類到子資料夾：
 - PDF 檔案 → 「文件/PDF」資料夾
 - Excel/CSV 檔案 → 「文件/試算表」資料夾
 - 圖片 (jpg, png, gif) → 「媒體/圖片」資料夾
 - 壓縮檔 (zip, rar) → 「壓縮檔」資料夾
 - 其他 → 「其他」資料夾
3. 如果目標資料夾不存在，自動建立
4. 搬移後紀錄每個檔案的新舊路徑到 log 檔案
5. 如果遇到同名檔案，在新檔名後面加上日期（不要覆蓋）

步驟 3：加入 AI 輔助的分類

請修改程式，讓 AI 幫忙處理無法依副檔名判斷的情況：

對於副檔名不明確的檔案，用 AI 根據檔名猜測應該放到哪個資料夾。

例如：「2025Q2銷售分析.dat」→ AI 判斷可能是試算表類型

實作二：PDF 發票批次資料提取

步驟 1：安裝 PDF 工具

```
pip install pdfplumber
```

步驟 2：向 GitHub Copilot 描述需求

請幫我寫一個 PDF 發票自動化提取程式：

1. 掃描「發票資料夾」內所有的 .pdf 檔案
2. 用 pdfplumber 提取每份 PDF 的文字內容
3. 將提取的文字傳給 OpenAI API，讓 AI 辨識並以 JSON 格式回傳：

```
{
  "廠商名稱": "",
  "發票號碼": "",
  "發票日期": "",
  "稅前金額": 0,
  "稅額": 0,
  "總金額": 0
}
```
4. 如果 AI 無法確定某個欄位，填入 "UNKNOWN"
5. 將所有提取結果匯整成一份 Excel 報表
6. 標記所有含有 "UNKNOWN" 的列，提示需要人工審核

實作三：傳統桌面系統自動化示範

步驟 1：安裝 UI 自動化工具

```
pip install pyautogui  
pip install pygetwindow
```

步驟 2：示範：自動操控「記事本」（基礎練習）

請幫我寫一個 pyautogui 程式：

1. 開啟 Windows 記事本
2. 等待記事本視窗出現
3. 在記事本中輸入今天的日期和「自動化測試成功」
4. 儲存檔案到指定路徑

步驟 3：進階：抓取老舊系統的螢幕資料

透過截圖 + AI Vision 的方式提取傳統系統中的資料：

請幫我寫一個程式：

1. 找到名為「XXX 系統」的視窗並切換到前景
2. 對該視窗截圖
3. 將截圖傳給 OpenAI Vision API，請 AI 提取畫面中的資料欄位
4. 將提取的資料存入 Excel

程式碼範例（虛擬碼）

資料夾自動整理

載入 pathlib, shutil, datetime 套件

設定路徑

來源資料夾 = 路徑("~/Downloads")

目標根資料夾 = 路徑("~/下載-已整理")

依副檔名對應目標資料夾

分類規則 = {

```
    ".pdf": "文件/PDF",  
    ".xlsx": "文件/試算表",  
    ".csv": "文件/試算表",  
    ".jpg": "媒體/圖片",  
    ".png": "媒體/圖片",  
    ".zip": "壓縮檔",  
    ".rar": "壓縮檔",
```

```
}
```

```

搬移紀錄 = []

# 遍歷所有檔案
對 每個檔案 在 來源資料夾 中（只看檔案，不看子資料夾）：
    副檔名 = 取得副檔名(檔案).轉小寫()

    如果 副檔名 在 分類規則 中：
        目標子資料夾 = 分類規則[副檔名]
    否則：
        目標子資料夾 = "其他"

    目標路徑 = 目標根資料夾 / 目標子資料夾
    建立資料夾(目標路徑, 含父資料夾=真, 若已存在不報錯=真)

# 處理同名衝突
最終目標 = 目標路徑 / 檔案.名稱
如果 最終目標 已存在：
    當前日期 = 取得當前日期字串()
    新檔名 = f"{檔案.主檔名}_{當前日期}{副檔名}"
    最終目標 = 目標路徑 / 新檔名

# 執行搬移
搬移(檔案, 最終目標)
搬移紀錄.加入({"原始路徑": 檔案, "新路徑": 最終目標})

# 寫入 log
將搬移紀錄寫入("整理紀錄.csv")
顯示 f"完成! 共整理 {len(搬移紀錄)} 個檔案"

```

PDF 資料提取

```

載入 pdfplumber, openai, pandas 套件

函式 提取PDF內容(pdf檔案路徑):
    使用 pdfplumber.open(pdf檔案路徑) 作為 pdf:
        全文 = ""
        對 每一頁 在 pdf.pages 中:
            全文 += 每一頁.extract_text()
        回傳 全文

函式 AI解析發票(原始文字):
    提示詞 = f"""
    以下是一份發票的文字內容:
    {原始文字}

    請提取以下欄位並以 JSON 格式回傳, 無法判斷的欄位請填 "UNKNOWN":
    {{
        "廠商名稱": "",
        "發票號碼": "",
        "發票日期": "",

```

```
"稅前金額": 0,
"稅額": 0,
"總金額": 0
}}
只回傳 JSON，不要說明。
"""

回應 = 呼叫LLM(提示詞)
回傳 解析JSON(回應)

# 批次處理
所有結果 = []
對 每個pdf檔案 在 列出資料夾("發票資料夾", "*.pdf") 中:
    顯示 f"處理中: {pdf檔案.名稱}"
    文字 = 提取PDF內容(pdf檔案)
    結果 = AI解析發票(文字)
    結果["原始檔案"] = pdf檔案.名稱
    所有結果.加入(結果)

# 輸出報表
報表 = 建立DataFrame(所有結果)
報表["需人工審查"] = 報表.任何欄位包含("UNKNOWN")
報表.寫入Excel("發票彙整.xlsx")
```

練習題 / 作業

課堂練習

練習 1：資料夾整理規則設計

針對你真實的工作資料夾，設計一套整理規則：

- 你希望依照哪些維度分類（類型、日期、專案名稱）？
- 對於不確定如何分類的檔案，希望 AI 如何處理？
- 整理後的資料夾結構長什麼樣？

練習 2：PDF 提取難度評估

找找看你手邊的 PDF 文件，判斷它是哪種類型：

- 可以複製文字的 → 文字型 PDF（容易處理）
- 無法複製文字的 → 掃描型 PDF（需要 OCR）

並思考：這份 PDF 的格式有多複雜？AI 能夠正確提取嗎？可能的困難點在哪？

課後作業

1. **資料夾整理程式**：用 GitHub Copilot 協助撰寫一個自動整理你電腦某個資料夾的腳本（可以先用測試資料夾）。
2. **PDF 提取實作**：找一份簡單的 PDF 文件（例如報名表或收據），嘗試用 pdfplumber 提取文字，再讓 AI 整理成結構化資料，觀察成功率和可能的問題。

延伸閱讀

檔案系統操作：

- Python pathlib 文件 — <https://docs.python.org/3/library/pathlib.html>
- watchdog — 資料夾監控套件 — <https://python-watchdog.readthedocs.io/>

PDF 處理：

- pdfplumber — <https://github.com/jsvine/pdfplumber>
- PyMuPDF 文件 — <https://pymupdf.readthedocs.io/>

桌面自動化：

- pyautogui 文件 — <https://pyautogui.readthedocs.io/>
- 《Automate the Boring Stuff with Python》— 第 18-20 章，桌面自動化相關章節

9/01 | 第07堂 | Vibe Coding 驅動：智慧動態報表系統開發

學習目標

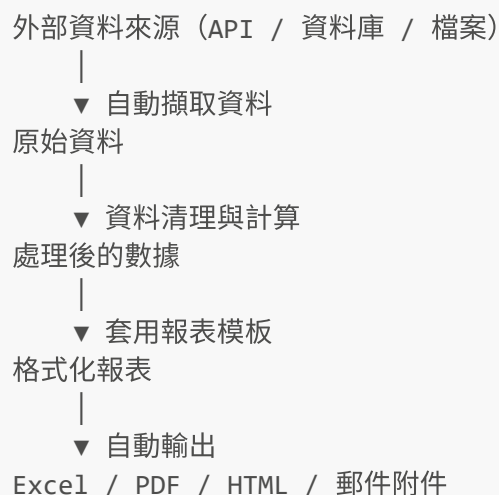
完成本堂課後，學員將能夠：

- 運用 Vibe Coding 實現從資料取得到報表生成的**端到端（End-to-End）**流程
- 整合外部公開 API，取得即時資料（匯率、股價、天氣等）
- 建立動態資料模型，讓報表自動隨資料更新
- 輸出格式多樣的報表（Excel、HTML、PDF）

核心概念說明

1. 什麼是「端到端報表系統」？

端到端（End-to-End）意味著整個流程從頭到尾完全自動化，不需要人工介入：



傳統 vs. 智慧動態報表的差異：

	傳統手工報表	智慧動態報表系統
資料來源	手動下載、複製貼上	自動從 API 或資料庫取得
更新頻率	需要人工觸發	可排程定時自動執行
人力需求	每次都需要人力介入	設置一次，持續自動運行
錯誤率	人工操作容易出錯	流程一致，錯誤可追蹤

2. API 的基礎概念

****API (Application Programming Interface, 應用程式介面) ****是讓不同軟體之間互相溝通的橋樑。

一個生活化的比喻：

在餐廳點餐時：

- **你** = 你的程式
- **菜單** = API 文件（說明可以點什麼、怎麼點）
- **服務生** = API
- **廚房** = 提供資料的服務

你不需要知道廚房怎麼運作，只需要按照菜單（API 文件）點餐（發送請求），服務生（API）就會把餐點（資料）送來。

REST API 的基本結構：

請求 (Request)：

方式：GET / POST / PUT / DELETE

網址：https://api.example.com/data?param=value

標頭：Authorization: Bearer YOUR_API_KEY

主體：{ "key": "value" } ← POST 請求才有

回應 (Response)：

狀態碼：200 成功 / 401 未授權 / 404 找不到 / 500 伺服器錯誤

資料：{ "result": [...] } ← 通常是 JSON 格式

常用的公開 API (適合新手練習)：

API 名稱	提供的資料	是否需要 Key
ExchangeRate-API	匯率資料	是 (免費)
OpenWeatherMap	天氣資料	是 (免費)
CoinGecko	加密貨幣價格	否 (部分免費)
REST Countries	各國資訊	否

API 名稱	提供的資料	是否需要 Key
政府資料開放平台	台灣各類公開資料	否

3. 動態資料建模

靜態報表：資料是固定的，報表也是固定的。**動態報表：**資料隨時間變化，報表自動根據最新資料計算和呈現。

動態報表的三個核心要素：

1. **資料層：**從哪裡取得資料？取得頻率？
2. **計算層：**需要做哪些計算（加總、平均、同比增長）？
3. **呈現層：**如何顯示（表格、圖表、關鍵指標）？

一個業務日報表的動態模型：

計算「今日業績」：

今日銷售 = 從資料庫取得今日所有訂單金額

目標業績 = 從設定檔讀取本月目標 / 本月天數

達成率 = 今日銷售 / 目標業績 * 100%

與昨日比較 = (今日銷售 - 昨日銷售) / 昨日銷售 * 100%

呈現方式：

KPI 卡片：今日業績 / 達成率（顏色：達成=綠色，未達成=紅色）

長條圖：本月每日業績趨勢

排行榜：今日業務員業績排名 Top 10

4. 報表輸出格式選擇

格式	工具	適合情境
Excel (.xlsx)	openpyxl、xlsxwriter	給需要進一步分析資料的人
HTML	Jinja2 + matplotlib	在瀏覽器或郵件中直接檢視
PDF	reportlab、weasyprint	正式文件、合約、對外報告
PowerPoint (.pptx)	python-pptx	管理報告、簡報用途

5. Vibe Coding 在複雜系統中的應用策略

當需要構建一個複雜的系統（如完整的報表生成系統），Vibe Coding 的策略是：

「分而治之」原則：不要一次把整個系統描述給 AI，而是**拆解成小模組**，逐一生成和測試，再組合在一起。

複雜需求：「一個自動產生每日業務報表的系統」

拆解為：

- 模組 1：資料取得函式（呼叫 API）
- 模組 2：資料清理函式（處理缺失值、格式化）
- 模組 3：指標計算函式（加總、排名、增長率）
- 模組 4：報表樣式函式（設定 Excel 格式、圖表）
- 模組 5：主程式（組合以上模組，設定排程）

每個模組先單獨開發和測試，最後整合。

實作步驟（逐步教學）

實作一：呼叫公開 API 取得即時資料

步驟 1：向 GitHub Copilot 描述第一個子模組

請幫我寫一個 Python 函式，呼叫 ExchangeRate-API 取得即時匯率：

1. 函式名稱：get_exchange_rates(base_currency="TWD")
2. API Key 從環境變數讀取
3. 回傳一個字典，包含主要貨幣對台幣的匯率（USD、EUR、JPY、CNY、HKD）
4. 加入錯誤處理：API 呼叫失敗時回傳 None 並記錄錯誤
5. 加入簡單的快取機制：如果 1 小時內已經呼叫過，回傳快取結果

步驟 2：測試這個函式

1. 直接呼叫函式，確認能正確取得匯率資料
2. 設定錯誤的 API Key，確認錯誤處理正常運作

實作二：建立 Excel 動態報表

步驟 2：向 Copilot 描述報表生成模組

請幫我寫一個 Excel 報表生成函式，輸入是匯率資料字典，輸出是一份格式化的 Excel 報表：

1. 第一個工作表「匯率快照」：
 - 顯示今日日期
 - 每個貨幣對一行，顯示：幣別、匯率、與上次比較的變化（漲/跌）
 - USD、EUR 欄加上適當格式（2位小數）、JPY 用0位小數
 - 若匯率比前次上漲，字體設為紅色；若下跌，設為綠色
2. 第二个工作表「圖表」：
 - 插入一個長條圖，顯示各幣別換算 NT\$100 可得多少外幣
3. 報表儲存檔名格式：「匯率報表_YYYYMMDD.xlsx」

實作三：整合為完整的端到端流程

步驟 1：向 Copilot 描述主程式邏輯

請幫我整合剛才開發的函式，建立一個完整的自動化報表系統：

1. 主程式執行時：
 - a. 從多個 API 取得資料（匯率 + 今日天氣，兩個資料都要錯誤處理）
 - b. 生成 Excel 報表
 - c. 生成一份 HTML 摘要（只有關鍵指標，容易用郵件傳送）
 - d. 顯示「報表生成成功！」和檔案路徑
2. 加入執行時間記錄：顯示每個步驟花了多少秒
3. 所有設定（API Key、輸出資料夾）都從環境變數讀取

程式碼範例（虛擬碼）

API 資料取得函式

載入 requests, json, datetime, os 套件

快取 = {} # 存放暫時的快取資料

函式 取得匯率(基準貨幣="TWD"):

檢查快取

快取鍵 = f"匯率_{基準貨幣}"

如果 快取鍵 在 快取 中:

快取時間 = 快取[快取鍵]["取得時間"]

如果 距今不超過 60 分鐘:

回傳 快取[快取鍵]["資料"]

呼叫 API

API_KEY = 從環境變數讀取("EXCHANGE_RATE_API_KEY")

API_URL = f"https://v6.exchangerate-api.com/v6/{API_KEY}/latest/{基準貨幣}"

嘗試:

回應 = requests.get(API_URL, 逾時=10秒)

回應.確認狀態碼成功()

資料 = 解析JSON(回應)

結果 = {

"USD": 資料["conversion_rates"]["USD"],

"EUR": 資料["conversion_rates"]["EUR"],

"JPY": 資料["conversion_rates"]["JPY"],

...

}

快取[快取鍵] = {"資料": 結果, "取得時間": 現在時間() }

回傳 結果

如果發生 連線錯誤:

記錄錯誤("API 連線失敗")

回傳 None

Excel 動態報表生成

```

載入 openpyxl 套件
載入 openpyxl.styles (用於設定格式)

函式 生成匯率報表(今日匯率, 昨日匯率=None):
    活頁簿 = 建立新的Excel活頁簿()
    工作表 = 活頁簿.取得活動工作表()
    工作表.名稱 = "匯率快照"

    # 寫入標題列
    標題 = ["幣別", "今日匯率", "變動", "變動方向"]
    工作表.寫入第一列(標題)
    設定標題列格式(粗體, 背景色=灰色)

    # 寫入資料
    對 幣別, 匯率 在 今日匯率.items():
        變動 = 計算變動(今日匯率[幣別], 昨日匯率[幣別]) if 昨日匯率 else 0
        方向 = "▲" if 變動 > 0 else "▼" if 變動 < 0 else "-"
        顏色 = 紅色 if 變動 > 0 else 綠色 if 變動 < 0 else 黑色

        新列 = [幣別, 匯率, 變動, 方向]
        工作表.加入新列(新列)
        設定最後列字體顏色(顏色)

    # 插入圖表
    圖表 = 建立長條圖()
    圖表.資料來源 = 工作表["B2":"B末行"]
    圖表.類別資料 = 工作表["A2":"A末行"]
    圖表.標題 = "各幣別今日匯率"
    工作表插入圖表(圖表, 位置="E2")

    # 儲存
    今日日期 = 取得今日日期字串()
    活頁簿.儲存(f"匯率報表_{今日日期}.xlsx")
    顯示 "報表已儲存"

```

練習題 / 作業

課堂練習

練習 1: API 使用場景分析

思考你的工作場景，找出 2-3 個可以「透過 API 自動取得資料」的需求：

需求描述	可能可用的 API	資料更新頻率

練習 2: 報表設計

設計一份你工作中需要的「動態報表」：

- 報表的主題是什麼？
- 資料來源有哪些（手動填寫 / 資料庫 / API）？
- 需要計算哪些指標？
- 希望呈現什麼圖表？
- 報表的讀者是誰？需要什麼格式？

課後作業

1. **API 實作：**選一個公開 API（建議從「政府資料開放平台」找有興趣的資料集），用 GitHub Copilot 協助你撰寫一個能取得並顯示資料的程式。
2. **報表生成：**在 API 資料取得成功後，嘗試將資料輸出成一份 Excel 報表（可以只有資料，不需要太複雜的格式）。

延伸閱讀

API 學習資源：

- requests 套件文件 — <https://requests.readthedocs.io/>
- 台灣政府資料開放平台 — <https://data.gov.tw/>
- Postman（API 測試工具） — <https://www.postman.com/>

報表生成工具：

- openpyxl 文件 — <https://openpyxl.readthedocs.io/>
- xlswriter（另一個 Excel 工具，圖表功能更強） — <https://xlswriter.readthedocs.io/>
- Jinja2（HTML 模板引擎） — <https://jinja.palletsprojects.com/>

資料視覺化：

- matplotlib — <https://matplotlib.org/>
- plotly（互動式圖表） — <https://plotly.com/python/>

9/08 | 第08堂 | 實作：智慧多通路內容行銷與分發系統

學習目標

完成本堂課後，學員將能夠：

- 設計並封裝 AI 代理的指令集，讓 Agent 能可靠地執行內容生成任務
- 使用 Prompt Engineering 技巧，讓 AI 針對不同平台產出適合的行銷內容
- 建立自動化的多平台內容分發 workflow
- 了解各平台 API 的串接方式與使用限制

核心概念說明

1. 多通路內容行銷的挑戰

現代行銷需要同時在多個平台上發布內容：

平台	限制與特性
Facebook	長文適合，支援圖片、影片
Instagram	視覺優先，需高品質圖片，文字相對次要
LinkedIn	專業語氣，適合 B2B 內容，長文效果好
X (Twitter)	280 字元限制，短而有力
LINE 官方帳號	台灣本地化，需要本地語氣，支援彈性選單
電子郵件電子報	可包含詳細資訊，需要吸引人的主旨

手動管理多平台的問題：

- 每個平台需要重新改寫適合的版本，費時費力
- 容易發布風格不一致的內容
- 人工失誤（忘記發佈、發錯版本）頻率高

2. Prompt Engineering 用於內容生成

****Prompt Engineering（提示詞工程）****是設計和優化給 AI 的指令，讓 AI 產出更準確、更符合需求的輸出。

控制輸出品質的關鍵 **Prompt** 要素：

元素	說明	範例
角色設定	告訴 AI 它扮演什麼角色	「你是一位有 10 年經驗的品牌行銷專家」
目標受眾	描述內容的目標讀者	「目標受眾是 30-40 歲的台灣中階主管」
平台特性	說明發布平台的規則和風格	「這是 LinkedIn 貼文，語氣要專業，可以長一點」
內容要素	指定必須包含的資訊	「必須包含產品名稱、三個主要優點、行動號召」
格式要求	指定輸出的格式	「字數限制 200 字，要有 3 個 hashtag」
語氣規範	描述希望的寫作風格	「語氣要活潑友善，不要太制式」

針對不同平台的內容改寫策略：

```
# 輸入：一份產品資訊文件
產品名稱 = "SmartFlow AI 方案"
核心賣點 = ["節省 70% 人工時間", "無需寫程式", "3 分鐘上手"]
目標行動 = "立即申請免費試用"

# 讓 AI 改寫為各平台版本：

Facebook 版：
    語氣友善、故事化，字數 150-300 字，加入 emoji
```

LinkedIn 版：

專業商務語氣，說明商業價值，字數 200-400 字，
包含數據證明，加入 2-3 個相關 hashtag

X 版：

濃縮核心訊息，280 字元限制，1 個強烈的 CTA

Email 主旨：

吸引人點擊、製造好奇心，50 字元以內

3. AI 指令集封裝 (Function/Tool 設計)

當 Agent 執行複雜任務時，需要將整個工作流拆解為**可重複使用的工具函式**，再由 Agent 根據任務需求選擇調用哪些工具。

內容行銷 Agent 的工具清單設計：

工具 1：生成平台內容(產品資訊，平台名稱，語氣)

→ 輸出：該平台的貼文草稿

工具 2：調整內容長度(文字，目標字數)

→ 輸出：調整後的文字

工具 3：加入 Hashtag(文字，數量，主題類別)

→ 輸出：加上 hashtag 的文字

工具 4：生成圖片描述(產品資訊，平台風格)

→ 輸出：適合的圖片描述（可交給設計師或 AI 生圖工具）

工具 5：排程發布(平台，內容，發布時間)

→ 觸發對應平台 API 進行發布

工具 6：記錄發布結果(平台，發布時間，成功/失敗)

→ 儲存到發布記錄 Excel

4. 平台 API 串接概念

各社群平台都提供官方 API 供開發者使用，但各自有不同的規範：

常用平台 API 概覽：

平台	API 名稱	申請難度	主要限制
Facebook/Instagram	Meta Graph API	中（需審核）	每日發文次數限制
LinkedIn	LinkedIn Marketing API	中（需申請）	需要 Page 管理員權限
X (Twitter)	X API v2	低（基本免費）	免費版有讀寫限制

平台	API 名稱	申請難度	主要限制
LINE 官方帳號	LINE Messaging API	低	需要在台灣申請官帳

API 串接的通用流程：

1. 到平台的開發者中心申請帳號
2. 建立應用程式 (App)，取得 API Key 和 Secret
3. 完成 OAuth 2.0 授權流程，取得 Access Token
4. 使用 Access Token 呼叫 API 發布內容
5. 處理 API 的回應 (成功/失敗/限流)

自動排程發布的概念：

觀念：不是立即發布，而是依照最佳時間排程

研究顯示的最佳發布時間 (參考)：

Facebook: 週三 11:00, 13:00
Instagram: 週二-週五 10:00-14:00
LinkedIn: 週二-週四 07:00-08:00, 17:00-18:00
X: 週一-週五 09:00, 12:00, 17:00

實作步驟 (逐步教學)

實作一：建立多平台內容生成引擎

步驟 1：設計各平台的 Prompt 模板

向 GitHub Copilot 描述：

請幫我建立一個「多平台內容生成器」，讀取一份產品資訊的 JSON，並針對以下 4 個平台分別生成適合的貼文草稿：

1. Facebook (200-300字，親切風格，含 emoji，含行動號召)
2. LinkedIn (200-350字，專業商務語氣，含數據，2-3個hashtag)
3. X/Twitter (100字以內，精煉，含1個hashtag，1個行動號召)
4. 電子郵件主旨 (50字以內，吸引人點擊)

輸入格式 (JSON)：

```
{  
  "product_name": "產品名稱",  
  "features": ["特點1", "特點2", "特點3"],  
  "target_audience": "目標受眾描述",  
  "cta": "行動號召文字"  
}
```

輸出格式：每個平台一個字典，包含「草稿」和「字數」

步驟 2：測試生成品質

準備 2-3 個不同類型的產品資訊作為測試輸入，評估 AI 生成的內容：

- 語氣是否符合平台風格？
- 字數是否在限制內？
- 是否包含所有必要資訊？

實作二：加入人工審核流程

步驟 1：設計「草稿審核介面」

請幫我建立一個互動式的草稿審核程式：

1. 依序顯示每個平台的內容草稿
2. 對使用者詢問：a) 核准發布 b) 修改後發布 c) 跳過
3. 如果選擇「修改」，讓使用者輸入修改需求，AI 重新生成
4. 將所有核准的草稿儲存到 `approved_drafts.json` 檔案
5. 生成一份「發布計畫表」Excel，記錄各平台、草稿內容、預計發布時間

實作三：串接 LINE Notify（示範 API 發布流程）

本實作以 LINE Notify 為範例（申請最簡單），示範 API 串接的完整流程。

步驟 1：申請 LINE Notify Token

1. 前往 https://notify-bot.line.me/zh_TW/
2. 登入 LINE 帳號
3. 點選「個人頁面」→「發行存取權杖（開發人員用）」
4. 複製取得的 Token，設定為環境變數

步驟 2：向 Copilot 描述 API 串接需求

請幫我用 Python 寫一個函式，透過 LINE Notify API 發送訊息：

1. 函式名稱：`send_line_notification(message)`
2. LINE Notify Token 從環境變數 `LINE_NOTIFY_TOKEN` 讀取
3. 呼叫 `https://notify-api.line.me/api/notify` 傳送訊息
4. 加入錯誤處理：401 = Token無效、429 = 超出限制，分別給出明確的提示
5. 發送成功後回傳 `True`，失敗回傳 `False`

程式碼範例（虛擬碼）

多平台內容生成函式

函式 生成多平台內容(產品資訊):

```
平台設定 = {
  "Facebook": {
    "字數限制": "200-300字",
    "風格": "親切友善, 含適量 emoji",
    "必要元素": "產品特點 + 故事/情境 + 行動號召"
  },
  "LinkedIn": {
    "字數限制": "200-350字",
    "風格": "專業商務語氣",
    "必要元素": "商業問題 → 解決方案 → 數據佐證 → CTA + hashtag"
  },
  "X": {
    "字數限制": "100字以內",
    "風格": "精煉有力",
    "必要元素": "核心訊息一句話 + 1個hashtag + 連結"
  },
  "Email主旨": {
    "字數限制": "50字以內",
    "風格": "製造好奇心或緊迫感",
    "必要元素": "主題清楚 + 引發點擊動機"
  }
}
```

結果 = {}

對 每個平台, 設定 在 平台設定.items():

提示詞 = f"""

你是一位專業的行銷文案撰寫師。

請為以下產品/服務撰寫 {平台} 的貼文:

產品名稱: {產品資訊["product_name"]}

主要特點: {產品資訊["features"]}

目標受眾: {產品資訊["target_audience"]}

行動號召: {產品資訊["cta"]}

要求:

- 字數: {設定["字數限制"]}
- 語氣風格: {設定["風格"]}
- 必須包含: {設定["必要元素"]}
- 使用繁體中文

只回傳貼文草稿, 不要說明。

"""

草稿 = 呼叫LLM(提示詞)

結果[平台] = {

"草稿": 草稿,

"字數": 計算字數(草稿)

}

回傳 結果

發布計畫表生成

載入 pandas, openpyxl, datetime 套件

函式 生成發布計畫表(已核准草稿, 輸出檔案="發布計畫表.xlsx"):

```
資料 = []
發布時間計畫 = {
    "Facebook": "週三 11:00",
    "LinkedIn": "週二 08:00",
    "X": "週一-週五 12:00",
    "LINE": "週三 18:00"
}

對 每個平台, 內容 在 已核准草稿.items():
    資料.加入({
        "平台": 平台,
        "草稿內容": 內容["草稿"],
        "字數": 內容["字數"],
        "建議發布時間": 發布時間計畫.get(平台, "待定"),
        "狀態": "待發布",
        "發布者": "",
        "備註": ""
    })

發布計畫 = 建立DataFrame(資料)
發布計畫.寫入Excel(輸出檔案)
顯示 f"發布計畫表已生成: {輸出檔案}"
```

練習題 / 作業

課堂練習

練習 1: Prompt 差異測試

使用同一份產品資訊, 用以下三種不同的 Prompt 生成 Facebook 貼文, 比較差異:

1. 只提供產品名稱和特點 (最少資訊)
2. 加入目標受眾和語氣要求
3. 加入角色設定、目標受眾、語氣、字數、必要元素 (完整 Prompt)

觀察: Prompt 越完整, 輸出品質有何不同?

練習 2: 受眾定位

思考你的工作場景, 選一個你想行銷的產品或服務, 填寫:

- 產品名稱:
- 核心賣點 (3個):

- 目標受眾：
- 希望讀者做什麼（行動號召）：
- 哪些平台最適合這個產品？

課後作業

1. **內容生成實測：**將你在課堂練習 2 填寫的產品資訊，輸入到課堂建立的多平台生成器，評估每個平台草稿的品質，記錄哪些地方需要改進，以及如何修改 Prompt 來提升品質。
2. **LINE Notify 測試：**完成 LINE Notify 的申請和 API 串接，成功發送一條測試訊息到你的 LINE。

延伸閱讀

Prompt Engineering：

- OpenAI Prompt Engineering 指南 — <https://platform.openai.com/docs/guides/prompt-engineering>
- Anthropic Claude Prompt 設計最佳實務 — <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

平台 API：

- Meta Graph API — <https://developers.facebook.com/docs/graph-api/>
- LINE Notify — https://notify-bot.line.me/zh_TW/
- X API v2 — <https://developer.x.com/en/docs/x-api>

行銷自動化概念：

- 《Contagious》by Jonah Berger — 理解什麼樣的內容容易傳播
- HubSpot 部落格 — 內容行銷最佳實務（可搜尋「HubSpot blog content marketing」）

9/15 | 第09堂 | 實作：AI 智慧研報與商務情報搜集系統

學習目標

完成本堂課後，學員將能夠：

- 建立自動化的資料檢索與分析工作流
- 整合多個資料來源（新聞、財經網站、公開資料庫）進行情報搜集
- 使用 AI 將大量非結構化情報彙整為有商業洞察的研報
- 輸出格式化的分析文案與報告文件

核心概念說明

1. 什麼是商務情報（Business Intelligence）？

****商務情報（BI）****是指透過有系統地蒐集、分析和解讀資訊，幫助企業做出更好的決策。

情報搜集的典型應用場景：

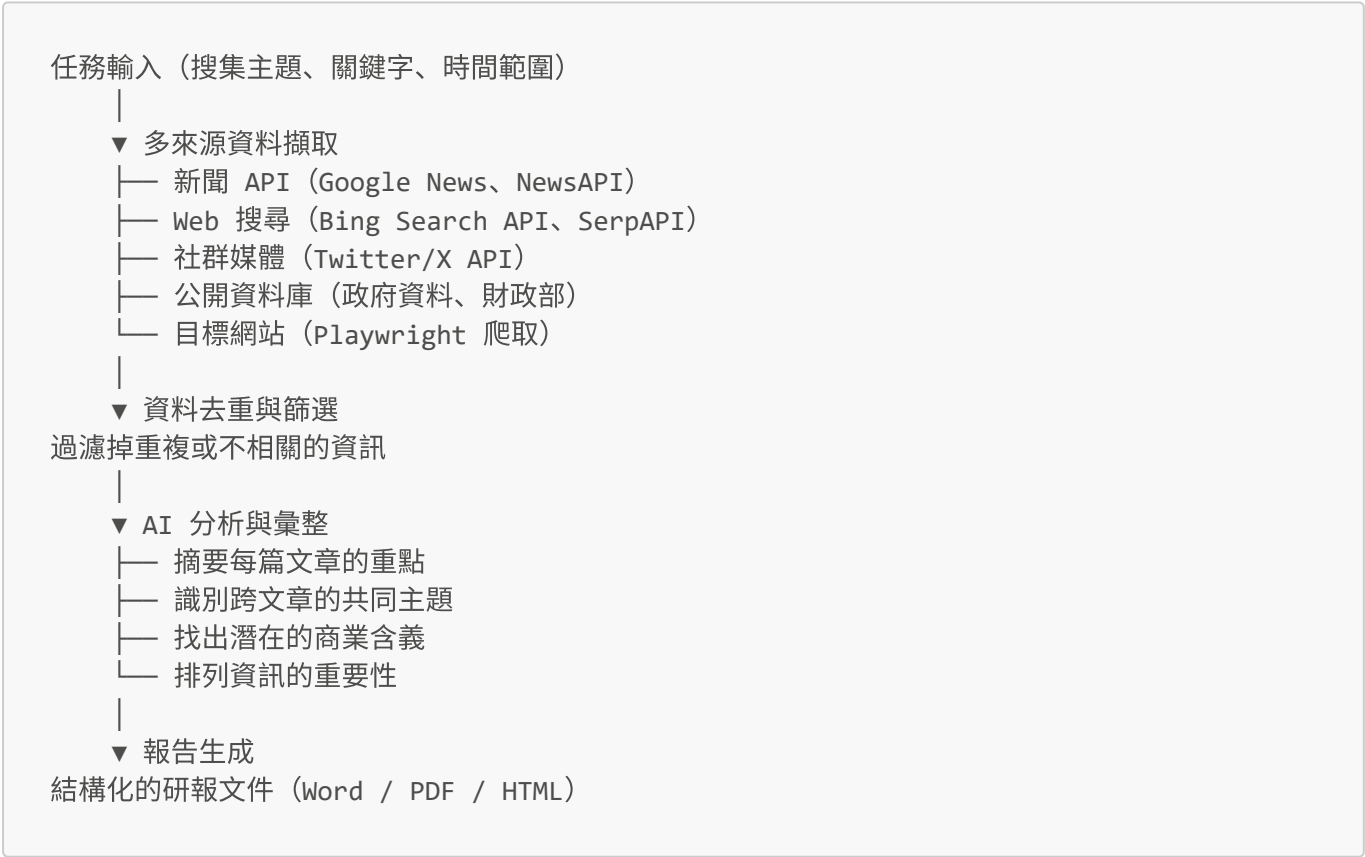
場景	需要搜集的資訊	使用目的
競品分析	競爭對手的定價、新聞、人才動向	調整自身策略
市場研究	產業趨勢、消費者輿論、法規動向	投資或進入新市場的決策
供應商評估	廠商公司資訊、財務狀況、客戶評價	選擇可靠的合作夥伴
客戶洞察	客戶公司近期動態、新聞、財報	銷售前準備、個人化服務

傳統情報搜集 vs. AI 自動化：

	傳統方式	AI 自動化
耗時	數小時到數天	數分鐘
覆蓋範圍	有限（人力限制）	可掃描大量來源
品質一致性	因人而異	格式一致
即時性	有延遲	可近乎即時

2. 自動化資料檢索的技術架構

情報搜集 Agent 的資料流：



NewsAPI 的基本用法：

請求參數：

關鍵字 (q)：要搜尋的主題

語言 (language)：zh (中文)、en (英文)

從此日期 (from)：搜尋的開始日期

排序方式 (sortBy)：publishedAt (最新)、relevancy (相關度)

回應格式：

```
[
  {
    "title": "文章標題",
    "source": {"name": "媒體名稱"},
    "publishedAt": "發布時間",
    "description": "文章摘要",
    "url": "原文連結"
  },
  ...
]
```

SerpAPI (搜尋引擎結果 API)：

SerpAPI 可以以程式方式取得 Google 搜尋結果，適合需要更廣泛資料來源的情境。

4. 研報的結構設計

一份有商業價值的研報通常包含以下結構：

[研報標題]

[搜集時間 / 分析範圍]

一、執行摘要 (Executive Summary)

- 3-5 條最重要的發現
- 主要結論與建議

二、主題概覽

- 背景說明
- 本次分析範圍

三、資料來源分析

- 共蒐集幾篇文章 / 幾個來源
- 主要媒體分佈
- 時間分佈

四、核心發現 (依重要性排列)

- 發現 1：[標題] + 分析 + 相關來源連結
- 發現 2：...
- 發現 3：...

五、趨勢分析
- 近期趨勢走向
- 潛在的機會與威脅
六、建議行動
- 短期（1-3 個月）可採取的行動
- 中長期策略方向
七、附錄：原始資料來源清單

5. AI 分析的 Prompt 設計

研報的核心價值在於「洞察（Insight）」，而不只是「摘要（Summary）」。

摘要 vs. 洞察的差別：

	摘要	洞察
說明什麼	「這篇文章說了 A, B, C」	「綜合三篇報導，可以發現 XX 公司正在做 Y，這意味著市場將往 Z 方向演變」
價值	省去閱讀原文的時間	提供決策依據

設計「洞察型」Prompt 的技巧：

不好的 Prompt：

「請摘要以下新聞文章。」

好的 Prompt：

「以下是過去一週關於『電動車電池市場』的 10 篇新聞。
請以一位產業分析師的角度，分析：

- 這些新聞反映出什麼重要的市場趨勢？
- 有哪些企業的動作值得特別關注？為什麼？
- 這對正在考慮進入這個市場的公司意味著什麼機會或風險？

請提供具體的、可行動的建議，不要只是描述事實。」

實作步驟（逐步教學）

實作一：建立新聞搜集模組

步驟 1：申請 NewsAPI Key

- 1. 前往 <https://newsapi.org/> 申請免費帳號
- 2. 取得 API Key，設定為環境變數 `NEWS_API_KEY`

步驟 2：向 GitHub Copilot 描述需求

請幫我用 Python 建立一個新聞搜集模組：

1. 函式名稱：fetch_news(keywords, days=7, language="zh", max_results=20)
2. 使用 NewsAPI 搜尋指定關鍵字在最近 N 天的新聞
3. 對每篇新聞回傳：標題、媒體名稱、發布時間、內文摘要、連結
4. 過濾掉：重複的標題、內容為 None 的文章
5. 儲存結果到一個 JSON 檔案（以關鍵字和日期命名）
6. API Key 從環境變數讀取

步驟 3：測試搜集結果

搜尋一個你工作相關的產業主題，檢視：

- 取得了多少篇文章？
- 有沒有重複或不相關的結果？
- 資料的時間分佈如何？

實作二：建立 AI 分析與洞察引擎

步驟 1：設計洞察分析 Prompt

請幫我建立一個分析函式：

1. 輸入：一個包含多篇新聞摘要的清單（每篇有標題、摘要、時間）
2. 分析主題：從函式參數傳入（例如：「台灣半導體產業」）
3. 讓 AI 以產業分析師角度輸出：
 - a. 執行摘要（3-5個重點，條列式）
 - b. 三個最重要的市場觀察（附說明）
 - c. 潛在的機會與風險各 2 條
 - d. 建議企業短期可採行的 2 個行動
4. 如果文章太多超過 Token 限制，分批處理後再彙整
5. 回傳結構化的字典格式

實作三：生成完整研報文件

步驟 1：安裝 Word 文件生成套件

```
pip install python-docx
```

步驟 2：向 Copilot 描述報告生成需求

請幫我建立一個研報文件生成函式：

1. 輸入：搜集到的新聞資料 + AI 分析結果
2. 輸出：一份格式完整的 Word 文件（.docx）
3. 文件結構：

- 封面：主題名稱、分析日期、資料來源筆數
 - 執行摘要
 - 核心發現（每項有標題、AI分析、原始來源連結）
 - 趨勢分析與建議
 - 附錄：原始資料清單
4. 格式要求：
- 標題使用 Heading 1/2/3 樣式
 - 重點文字加粗
 - 頁首顯示標題，頁尾顯示頁碼

程式碼範例（虛擬碼）

新聞搜集模組

載入 requests, json, datetime, os 套件

函式 搜集新聞(關鍵字清單, 天數=7, 語言="zh", 最多篇數=20):

API_KEY = 從環境變數讀取("NEWS_API_KEY")

結束日期 = 今天()

開始日期 = 今天() - 天數天

所有文章 = []

已見過的標題 = 集合() # 用來去重

對 每個關鍵字 在 關鍵字清單 中:

嘗試:

```
    回應 = 呼叫NewsAPI(  
        q=關鍵字,  
        from=開始日期,  
        to=結束日期,  
        language=語言,  
        sortBy="relevancy",  
        pageSize=最多篇數  
    )
```

對 每篇文章 在 回應["articles"] 中:

如果 文章["description"] 為 None: 跳過

如果 文章["title"] 在 已見過的標題 中: 跳過 # 去重

已見過的標題.加入(文章["title"])

```
所有文章.加入({  
    "標題": 文章["title"],  
    "媒體": 文章["source"]["name"],  
    "時間": 文章["publishedAt"],  
    "摘要": 文章["description"],  
    "連結": 文章["url"]  
})
```

如果發生錯誤:

記錄錯誤(關鍵字)

```
# 儲存到 JSON
檔案名 = f"新聞搜集_{關鍵字清單[0]}_{今天日期}.json"
寫入JSON(所有文章, 檔案名)
顯示 f"共搜集 {len(所有文章)} 篇文章"
回傳 所有文章
```

AI 洞察分析函式

函式 分析產業情報(文章清單, 分析主題):

```
# 組合所有文章摘要 (注意 Token 長度限制)
文章彙整 = ""
對 i, 文章 在 enumerate(文章清單):
    文章彙整 += f"""
        [{i+1}] {文章["標題"]} ({文章["媒體"]}, {文章["時間"]})
        {文章["摘要"]}
    """
```

分析提示詞 = f"""

你是一位資深產業分析師，請分析以下關於「{分析主題}」的新聞，
提供具有商業價值的洞察。

=== 新聞資料 ===

{文章彙整}

=== 新聞資料結束 ===

請以結構化方式輸出以下分析 (使用繁體中文):

1. 執行摘要 (3-5個最重要發現, 條列式)
2. 三大市場觀察 (每項: 觀察標題 + 2-3句分析說明)
3. 機會分析 (2條, 說明對企業的潛在機會)
4. 風險提示 (2條, 說明潛在風險)
5. 建議行動 (短期可採行的2項具體行動)

請確保分析具有洞察力，不要只是重述新聞內容。

"""

AI分析 = 呼叫LLM(分析提示詞)

回傳 AI分析

練習題 / 作業

課堂練習

練習 1: 情報需求盤點

找出在你的工作中，有哪些情報需求是**目前靠人工定期搜集**的：

需要搜集的情報	目前多久搜集一次	需要耗費多少時間	AI 能幫上的地方
---------	----------	----------	-----------

練習 2：Prompt 品質比較

針對同一組新聞文章，設計兩種不同的分析 Prompt：

- 版本 A：只要求「摘要新聞重點」
- 版本 B：以分析師角度要求「提供洞察與建議」

比較兩個版本的輸出差異，哪個對決策更有幫助？

課後作業

1. **情報搜集實作：**選一個你感興趣的產業或公司作為主題，使用 NewsAPI 搜集最近一週的相關新聞（至少 10 篇），然後讓 AI 生成一份簡短的情報摘要（不需要完整研報，500 字以內即可）。
2. **洞察品質評估：**閱讀 AI 生成的摘要，評估：
 - 哪些洞察是你看完新聞也能自己想到的？
 - 哪些洞察是 AI 幫你「串連」出來的、你可能沒注意到的？
 - 如何修改 Prompt 讓洞察更有深度？

延伸閱讀

資料來源 API：

- NewsAPI — <https://newsapi.org/docs>
- SerpAPI（Google 搜尋結果） — <https://serpapi.com/>

資訊提取技術：

- BeautifulSoup 爬蟲教學 — <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

報告生成：

- python-docx — <https://python-docx.readthedocs.io/>

商務情報概念：

- 《競爭優勢》by Michael Porter — 競爭情報分析的經典理論
- 搜尋「CI（Competitive Intelligence）情報分析方法」了解系統化情報分析框架

9/22 | 第10堂 | 實作：AI 智慧驅動視覺化簡報生成

學習目標

完成本堂課後，學員將能夠：

- 使用 AI 將長篇文稿或大綱自動轉化為結構化的簡報邏輯
- 使用 python-pptx 套件以程式方式生成 PowerPoint 簡報

- 自動化套用一致的設計風格至所有投影片
- 建立從「文字輸入」到「完整簡報輸出」的端到端自動化流程

核心概念說明

1. 為什麼簡報製作是 AI 自動化的理想場景？

製作一份簡報通常需要：

1. 閱讀大量資料 → **AI 最擅長的摘要與結構化**
2. 決定每張投影片的內容邏輯 → **AI 的架構規劃能力**
3. 確保視覺風格一致 → **程式操控 PPT 的強項**
4. 反覆調整格式和排版 → **程式比人工快、精準**

簡報製作因此是「AI 語意能力 + 程式自動化」結合效益最大的場景之一。

2. AI 語意架構化：從長文到投影片結構

核心任務： 將一份冗長的文稿或大綱，轉化為「每張投影片該說什麼」的清單。

好的投影片結構原則：

原則	說明
金字塔原則	結論先行，再提供支撐細節
一頁一訊息	每張投影片只傳達一個核心觀點
3的法則	能分成三點的，就分三點（容易記憶）
故事弧線	問題 → 影響 → 解決方案 → 行動

AI 架構化的 Prompt 策略：

輸入：一篇 1000 字的市場分析報告

需要 AI 完成的任務：

1. 識別報告的核心論點（1-2句）
2. 拆解為 8-12 張投影片的架構
3. 每張投影片提供：
 - 投影片標題（8字以內，有力、精準）
 - 投影片類型（標題頁、大綱、內容、數據、結論）
 - 主要內容（3-5個條列重點或1段文字）
 - 建議的視覺輔助（圖表類型、圖示建議）
4. 以 JSON 格式輸出，方便程式讀取

3. PowerPoint 自動化：python-pptx

python-pptx 是一個 Python 套件，可以用程式碼建立、修改 PowerPoint 檔案（.pptx）。

投影片的組成元素：

元素	說明
Slide（投影片）	每一頁投影片
Slide Layout（版面配置）	預設的排版模板（標題頁、內容頁等）
Placeholder（接受框）	版面配置中的文字或圖片位置
TextFrame（文字框）	包含文字的容器
Paragraph（段落）	文字框內的一個段落
Run（文字段）	段落中相同格式的一段文字
Shape（形狀）	矩形、圓形、線條等
Chart（圖表）	長條圖、折線圖、圓餅圖

設計一致性的實現方式：

一致設計的要素：

- 主題色彩：定義 3-5 個主色和輔助色
- 字型規範：標題字體 / 內文字體 / 大小規格
- 版面規則：每張投影片左側留白多少、標題位置固定

在 python-pptx 中：

- 使用 Presentation Theme 套用整體樣式
- 使用 Slide Master 統一管理母片設計
- 每張投影片繼承相同的基礎設計，再加入個別內容

4. 投影片類型與對應視覺設計

不同類型的投影片需要不同的版面設計：

投影片類型	適合的版面	主要元素
封面頁	全版面視覺 + 大標題	主標題、副標題、日期、機構
大綱頁	條列式清單	本次簡報的章節架構
內容頁（文字）	標題 + 條列點	3-5個重點（每點一行最佳）
數據頁	標題 + 圖表	一個清楚的圖表 + 簡短說明
比較頁	左右分欄	兩個選項並排比較
過渡頁	大字、少內容	章節標題，告訴觀眾接下來要講什麼
結論頁	重點摘要	3-5個最重要的帶走訊息

投影片類型	適合的版面	主要元素
行動號召頁	醒目視覺	明確的下一步行動

5. 完整的簡報生成流程



實作步驟（逐步教學）

實作一：AI 架構規劃模組

步驟 1：安裝套件

```
pip install python-pptx
```

步驟 2：向 GitHub Copilot 描述架構規劃函式

請幫我建立一個「投影片架構規劃」函式：

- 輸入：原始文稿（字串）、簡報主題、目標張數（預設 10）
- 呼叫 OpenAI API，讓 AI 設計投影片架構
- AI 回傳的格式必須是合法的 JSON，結構如下：

```
{  
  "title": "整份簡報的標題",  
  "slides": [  
    {
```

```
        "index": 1,
        "type": "cover/outline/content/data/transition/conclusion",
        "title": "投影片標題",
        "points": ["重點1", "重點2", "重點3"],
        "notes": "演講者備注"
    },
    ...
]
```

4. 如果 AI 回傳不是合法 JSON，要能重試（最多 3 次）

步驟 3：測試架構規劃

準備一段 500-800 字的文稿，執行架構規劃函式，觀察 AI 規劃的投影片架構是否合理。

實作二：PowerPoint 自動生成

步驟 1：設計投影片版面函式

請幫我用 python-pptx 建立一個 PPT 生成函式，以下是要求：

1. 一個主要函式：generate_presentation(slides_json, output_path)
2. 支援的投影片類型（以 slides_json 中的 type 欄位判斷）：
 - "cover"：封面（大標題置中，副標題，顯示今天日期）
 - "content"：內容頁（左上角標題，下方 3-5 個條列重點）
 - "transition"：過渡頁（章節標題置中，大字，背景色不同）
 - "conclusion"：結論頁（標題 + 重點摘要）
3. 所有投影片使用統一的設計規範：
 - 標題字型：微軟正黑體，32pt，黑色
 - 內文字型：微軟正黑體，20pt，深灰色
 - 主色調：深藍色（#1A3A5C），過渡頁用主色背景
 - 頁碼顯示在右下角（封面除外）

步驟 2：加入圖表功能（資料說明頁）

請幫我加入圖表支援：

如果投影片類型是 "data" 且 JSON 中有 chart_data 欄位：

- 建立一個長條圖或折線圖（依 chart_type 欄位決定）
- 將圖表嵌入投影片右側
- 左側顯示 2-3 個重點說明

實作三：整合完整流程

步驟 1：建立主程式

請幫我建立一個主程式，整合以上所有功能：

1. 讀取使用者輸入（從命令列或 JSON 設定檔）：
 - 輸入文稿路徑
 - 簡報主題
 - 投影片數量
 - 輸出 PPT 路徑
2. 呼叫架構規劃函式，取得 JSON 架構
3. 顯示規劃好的架構，詢問使用者是否確認（y/n）
4. 若確認，呼叫 PPT 生成函式
5. 生成完成後顯示成功訊息和輸出路徑

程式碼範例（虛擬碼）

AI 投影片架構規劃

函式 規劃投影片架構(文稿內容，簡報主題，目標張數=10)：

提示詞 = f"""

你是一位專業的簡報設計顧問。

請將以下文稿設計成 {目標張數} 張左右的投影片架構。

簡報主題：{簡報主題}

文稿內容：

{文稿內容}

請以 JSON 格式回傳，結構如下：

```
{{
  "title": "簡報總標題",
  "slides": [
    {{
      "index": 1,
      "type": "cover",
      "title": "...",
      "subtitle": "...",
      "points": [],
      "notes": "演講者備注"
    }},
    ...
  ]
}}
```

設計原則：

- 第一張必須是封面（cover 類型）
- 最後一張必須是結論或行動號召
- 每張投影片傳達一個清楚的重點
- 純文字的 points 每個條列不超過 20 字
- 只回傳 JSON，不要有其他說明文字

"""

對 重試次數 從 1 到 3：

嘗試：

AI回應 = 呼叫LLM(提示詞)

結果 = 解析JSON(AI回應)

回傳 結果

如果發生 JSON解析錯誤：

顯示 f"第 {重試次數} 次 JSON 解析失敗，重試中..."

拋出例外("無法取得合法的架構 JSON")

PowerPoint 投影片生成

載入 pptx 套件

載入 pptx.util (Inches, Pt, Emu)

載入 pptx.dml.color (RGBColor)

常數：

主色 = RGBColor(0x1A, 0x3A, 0x5C) # 深藍色

深灰色 = RGBColor(0x33, 0x33, 0x33)

白色 = RGBColor(0xFF, 0xFF, 0xFF)

標題字體 = "微軟正黑體"

內文字體 = "微軟正黑體"

函式 生成投影片(架構JSON, 輸出路徑):

簡報 = 建立新的Presentation()

對 每張投影片資訊 在 架構JSON["slides"] 中:

類型 = 投影片資訊["type"]

新投影片 = 簡報.加入新投影片()

如果 類型 == "cover":

新投影片.背景色 = 主色

加入文字框(新投影片, 投影片資訊["title"],
位置=置中, 字體=白色, 大小=40pt, 粗體=真)

如果 有副標題:

加入文字框(新投影片, 投影片資訊["subtitle"],
位置=副標題位置, 字體=白色半透明, 大小=24pt)

否則如果 類型 == "content":

加入標題(新投影片, 投影片資訊["title"], 顏色=主色)

對 每個重點 在 投影片資訊["points"] 中:

加入條列式文字(新投影片, 重點, 縮排=1)

否則如果 類型 == "transition":

新投影片.背景色 = 主色

加入置中大字(新投影片, 投影片資訊["title"], 顏色=白色, 大小=36pt)

加入頁碼 (封面除外)

如果 類型 != "cover":

加入頁碼(新投影片, 投影片資訊["index"])

```
簡報.儲存(輸出路徑)
顯示 f"簡報已儲存: {輸出路徑}"
```

練習題 / 作業

課堂練習

練習 1：投影片架構邏輯

給你以下一份短文，試著手動規劃成 5 張投影片的架構，再看看 AI 會怎麼規劃，比較差異：

台灣的中小企業面臨數位轉型的挑戰。根據調查，僅有 30% 的中小企業已完成基礎數位化。主要障礙包括：預算不足、人才短缺、對新技術不熟悉。然而，那些已完成轉型的企業平均提升了 25% 的生產力。解決方案包括：政府補助計畫、雲端工具導入、員工培訓計畫。未來三年內，不進行數位轉型的企業將面臨嚴峻的競爭劣勢。

練習 2：設計風格決策

思考以下場景，各需要什麼設計風格的簡報：

場景	受眾	適合的風格（正式/活潑/科技感）	色調建議
向投資人的募資簡報	投資人		
員工教育訓練	內部員工		
產品發表會	潛在客戶		
學術研究成果報告	學術圈		

課後作業

- 完整流程測試：**準備一份 600-1000 字的報告或文章（可以是工作相關的文件），執行本堂課建立的完整流程，生成一份 PPT，並評估輸出品質。
- 設計改進：**觀察生成的 PPT，找出 2-3 個設計上的不足，思考如何修改程式碼來改善。

延伸閱讀

PowerPoint 自動化：

- python-pptx 官方文件 — <https://python-pptx.readthedocs.io/>
- python-pptx 圖表教學 — 文件中的 Charts 章節

簡報設計原則：

- 《簡報聖經》by Garr Reynolds — 好簡報的設計原則
- 《金字塔原理》by Barbara Minto — 結構化思考與表達的經典
- TED 簡報技巧 — 搜尋「TED talk presentation tips」

AI 圖像生成（擴展閱讀）：

- DALL-E 3 API — 可整合至流程，自動生成簡報配圖
- Stable Diffusion — 開源圖像生成，可本地運行

9/29 | 第11堂 | 實作：智慧客服郵件決策支援系統

學習目標

完成本堂課後，學員將能夠：

- 建立基於 AI 語意理解的郵件自動分類與優先排序系統
- 模擬真實客服場景，自動生成符合情境的回覆草稿
- 設計複雜異議處理的決策樹，讓 Agent 能導航至正確的回覆策略
- 建立關鍵資訊自動標記機制，加速人工審核效率

核心概念說明

1. 客服自動化的商業價值

數字說話：

- 企業客服部門平均每天處理數百到數千封郵件
- 重複性問題（FAQ）通常佔來信量的 60-70%
- 人工閱讀並回覆每封郵件平均需要 5-15 分鐘

AI 可以解決的痛點：

痛點	AI 解決方案
來信量大、人力不足	自動分類 + 草稿生成，大幅縮短每封的處理時間
緊急信件淹沒在一般信件中	AI 自動識別優先級，確保緊急案件不被遺漏
回覆品質因人而異	AI 提供標準化草稿，確保品質一致
夜間或假日無法即時回覆	24小時自動發送初步響應，告知客戶已收到並將跟進

2. 郵件分類的維度設計

一個有效的客服郵件分類系統需要多個維度同時分析：

維度 1：意圖分類（Intent Classification）

意圖類型	說明	建議處理時限
產品詢問	詢問功能、規格、價格	24小時
訂單狀態	查詢訂單進度	4小時
退換貨申請	要求退款或換貨	2小時

意圖類型	說明	建議處理時限
投訴/抱怨	對產品或服務不滿	1小時
緊急問題	系統故障、安全問題	立即
讚美/感謝	正面回饋	48小時
垃圾郵件	廣告或無關信件	忽略

維度 2：情感分析（Sentiment Analysis）

情感程度	判斷依據	影響策略
強烈負面	語氣激烈、出現威脅或法律字眼	立即升級，主管介入
輕微負面	不滿但語氣平和	優先處理，同理心回覆
中性	無明顯情緒	標準處理
正面	感謝、讚美	標準回覆，可回覆感謝

維度 3：複雜程度

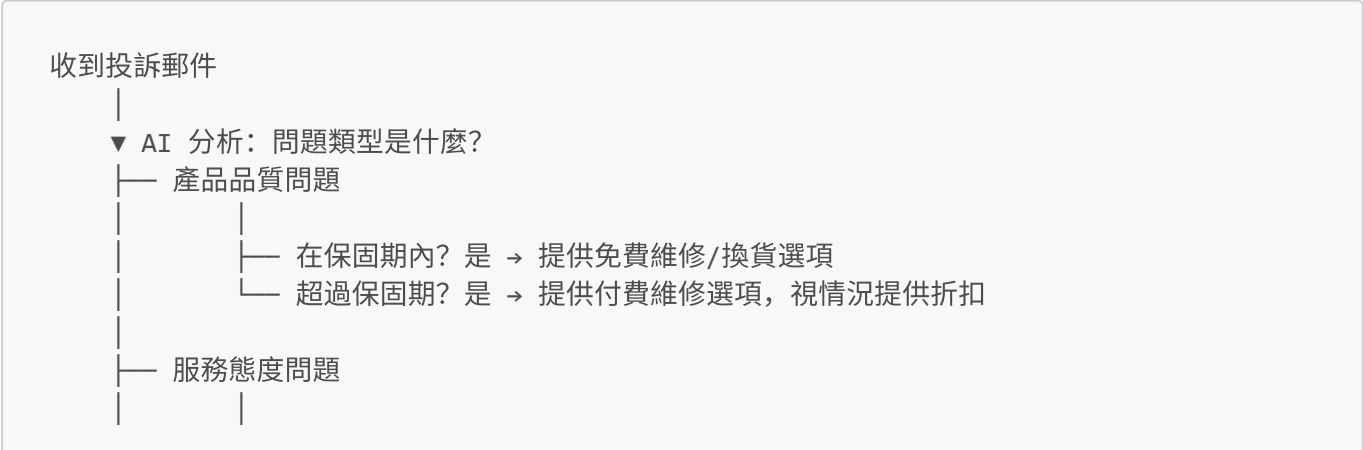
複雜程度	說明	處理方式
可自動化	標準問題，有固定答案	AI 直接生成回覆草稿
需人工判斷	個案處理，需查詢資料系統	AI 提供資訊，人工決策
需升級	超出一般客服權限	轉交主管或專責部門

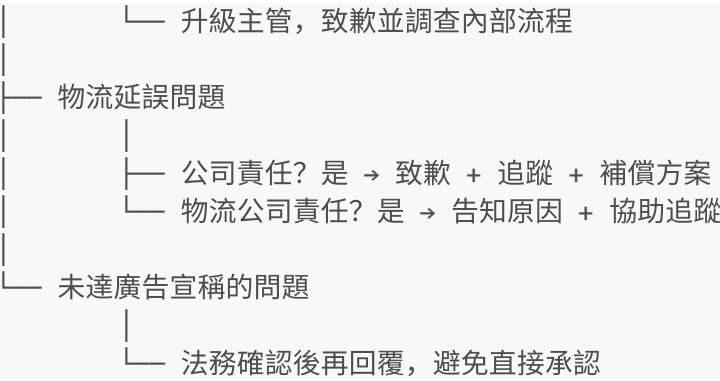
3. 異議處理的決策樹設計

****異議處理（Complaint Handling）****是客服中最具挑戰性的場景，需要：

- 1. 準確理解客戶的不滿點
- 2. 表達同理心
- 3. 提供可接受的解決方案
- 4. 確保客戶感到被重視

異議處理的決策流程：





4. 關鍵資訊自動標記

AI 在閱讀郵件後，應自動提取並標記以下關鍵資訊，讓人工審核者能快速掌握重點：

郵件摘要卡：

意圖：退換貨申請	優先級：高（●）
情感：強烈負面	複雜度：需人工
關鍵資訊：	
• 訂單號：#2025-07-1234	
• 購買日期：2025/07/10	
• 問題：產品有異味、懷疑品質問題	
• 客戶聲稱已拍照存證	
• 提及「若不處理將到消費者保護機構申訴」	
AI 建議：升級主管，24小時內必須回覆	

5. 資料安全與個資保護

客服郵件通常包含大量個人資料，在使用 AI 處理時需特別注意：

重要原則：

原則	說明
最小化傳送	只傳送 AI 分析所需的資訊，刪除不必要的個資
去識別化	在傳送前將姓名、電話等個資替換為代碼
記錄存取	記錄哪些郵件被 AI 處理，保留稽核軌跡
員工教育	確保所有使用人員了解個資保護規範
使用條款確認	確認 LLM 服務商的資料使用政策是否符合公司合規要求

實作步驟（逐步教學）

實作一：建立郵件分類引擎

步驟 1：設計分類 Prompt

向 GitHub Copilot 描述：

請幫我建立一個郵件分類函式：

1. 輸入：郵件主旨 + 郵件內文（限制內文最多 500 字，超過截斷）
2. 呼叫 LLM 進行以下分析，以 JSON 格式回傳：

```
{
  "intent": "產品詢問/訂單狀態/退換貨/投訴/緊急/讚美/垃圾郵件",
  "sentiment": "強烈負面/輕微負面/中性/正面",
  "priority": "立即/高/中/低",
  "complexity": "可自動化/需人工/需升級",
  "summary": "20字以內的摘要",
  "key_info": {
    "order_number": "訂單號（若有）",
    "product": "提到的產品（若有）",
    "main_issue": "核心問題描述",
    "escalation_signal": "有無升級警示（法律威脅/媒體威脅/個資要求等）"
  },
  "suggested_action": "建議的處理方式"
}
```
3. 個資保護：在傳給 LLM 之前，將郵件中的姓名替換為「[客戶]」、電話替換為「[電話]」

實作二：建立回覆草稿生成器

步驟 1：設計多情境的回覆 Prompt 庫

請幫我建立一個回覆草稿生成函式，需要支援以下情境：

1. 標準產品詢問回覆（友善、提供產品資訊、引導到購買）
2. 訂單狀態回覆（提供追蹤資訊的格式，請人工填入實際資料）
3. 退換貨投訴回覆（同理心開頭 + 標準處理流程 + 後續追蹤承諾）
4. 緊急問題初步回覆（確認收到 + 承諾後續跟進時間）
5. 讚美感謝回覆（簡短感謝 + 歡迎再次光臨）

要求：

- 語氣要專業但不失溫度
- 不要承諾任何具體的補償金額（這需要人工決定）
- 回覆要標記「[需人工填入的部分]」讓審核者清楚知道什麼地方需要確認
- 用繁體中文，語氣符合台灣商務慣例

實作三：整合完整的客服決策支援介面

步驟 1：建立處理流程主程式

請幫我整合以上功能，建立完整的客服支援工具：

1. 監控指定 Outlook 資料夾的新郵件（或讀取測試郵件清單）
2. 對每封郵件執行分類分析
3. 依優先級排序，高優先的排在最前面
4. 生成每封郵件的「摘要卡」（包含分類結果和關鍵資訊）
5. 對可自動化的郵件，生成回覆草稿並存入草稿箱
6. 對需升級的郵件，發送內部通知給主管（使用 LINE Notify 或郵件）
7. 將所有處理結果記錄到 Excel（含：處理時間、分類結果、是否有草稿）
8. 生成每日客服情況統計報表

程式碼範例（虛擬碼）

郵件分類函式（含個資保護）

載入 re（正規表達式）套件

函式 去除個資(郵件內文):

替換常見的個資格式

結果 = 郵件內文

結果 = 取代符合 "0[89]\d{8}" → "[電話]" # 手機號碼

結果 = 取代符合 "\d{4}-\d{4}" → "[電話]" # 市話格式

結果 = 取代符合 "身份證[字號:\s]*[A-Z]\d{9}" → "[身分證]"

結果 = 取代符合 "[A-Za-z0-9._%]+@[A-Za-z0-9._%]+\.[A-Za-z]+" → "[Email]"

回傳 結果

函式 分析郵件(主旨，內文):

個資保護：去除識別資訊

安全內文 = 去除個資(內文[:500]) # 限制500字

提示詞 = f"""

你是一個客服郵件分析助理。請分析以下郵件並以 JSON 格式回傳結果。

郵件主旨：{主旨}

郵件內文：{安全內文}

分析結果格式：

```
{{
  "intent": "產品詢問|訂單狀態|退換貨|投訴|緊急|讚美|垃圾郵件",
  "sentiment": "強烈負面|輕微負面|中性|正面",
  "priority": "立即|高|中|低",
  "complexity": "可自動化|需人工|需升級",
  "summary": "核心問題20字摘要",
  "key_info": {{
    "main_issue": "問題描述",
    "escalation_signal": true/false,
    "escalation_reason": "升級原因（若有）"
  }}
}},
```

```

        "suggested_action": "建議處理方式"
    }}
    只回傳 JSON。
    ""

    嘗試：
        AI回應 = 呼叫LLM(提示詞)
        回傳 解析JSON(AI回應)
    如果發生錯誤：
        記錄錯誤(主旨)
        回傳 {"priority": "中", "complexity": "需人工", "summary": "分析失敗，請人工
    審核"}

```

回覆草稿生成

```

回覆模板庫 = {
    "退換貨": ""
        感謝您聯繫我們。

        我完全理解您目前的困擾，非常抱歉我們的產品/服務讓您感到不便。

        針對您反映的 [主要問題]，我們將按照以下流程為您處理：
        1. [需人工填入：退換貨具體流程步驟]
        2. 預計處理時間：[需人工確認時間]
        3. [需人工填入：補償方案（若有）]

        我們將在 [需人工確認時間] 之前給您明確的回覆。
        若您有任何疑問，請隨時回覆此郵件。

        再次為給您造成的不便致上最誠摯的歉意。
    "",
    # ... 其他模板
}

函式 生成回覆草稿(郵件分析結果, 原始郵件摘要):
    意圖 = 郵件分析結果["intent"]
    情感 = 郵件分析結果["sentiment"]
    主要問題 = 郵件分析結果["key_info"]["main_issue"]

    如果 意圖 == "垃圾郵件":
        回傳 None # 垃圾郵件不回覆

    如果 郵件分析結果["complexity"] == "需升級":
        # 不自動生成回覆，轉人工
        通知主管(郵件分析結果)
        回傳 None

    # 使用 AI 生成個性化草稿
    提示詞 = f""
    請根據以下資訊，生成一封客服回覆郵件草稿：

```

```

客戶意圖：{意圖}
客戶情感狀態：{情感}
核心問題：{主要問題}

參考模板：
{回覆模板庫.get(意圖, 回覆模板庫["標準"])}

```

要求：

- 開頭需要針對客戶情感做適當的回應（負面情感需要先道歉/同理）
- 用繁體中文，語氣專業而溫暖
- 用 [需人工填入：說明] 標記所有需要人工確認的地方
- 不要承諾具體金額或時間，那些都要 [需人工填入]

"""

```

草稿 = 呼叫LLM(提示詞)
回傳 草稿

```

練習題 / 作業

課堂練習

練習 1：分類準確度測試

以下是五封假設的客服郵件，請先自行判斷分類，再讓 AI 分析，比較結果：

郵件摘要	你的判斷（意圖/優先級）	AI 的判斷	是否一致？
「請問你們有沒有適合 3 歲小孩的產品？」			
「我的訂單已經兩週了還沒收到！」			
「產品收到了，真的很好用，謝謝你們！」			
「如果這週內不解決，我要去找消基會！」			
「發票要抬頭打公司行號，請問怎麼操作？」			

練習 2：回覆草稿評分

針對課堂生成的一份回覆草稿，用以下標準評分（1-5分）：

- 同理心表達是否到位？
- 語氣是否專業？
- 是否清楚說明了處理方式？
- [需人工填入] 的標記是否在該標記的地方？

課後作業

1. **客服流程設計：**設計你的工作場景中（或假設一個行業）的完整客服郵件處理流程，包含：分類維度、各類型的回覆模板、升級規則、每日報表的格式。

2. **個資保護機制**：識別你設計的流程中，哪些資訊會傳送給外部 AI API，並說明如何在傳送前做去識別化處理。

延伸閱讀

客服自動化：

- Zendesk 部落格 — 客服 AI 最佳實務（搜尋「Zendesk AI customer service best practices」）
- 《客戶成功》— 了解客服文化與客戶體驗管理

個資保護：

- 個人資料保護法（台灣）— 了解台灣的個資法規範
- GDPR 概念介紹 — 如果業務涉及歐洲客戶

情感分析技術：

- HuggingFace 中文情感分析模型 — <https://huggingface.co>（搜尋「Chinese sentiment analysis」）
- VADER 情感分析（英文）— 快速、輕量的情感分析工具

10/06 | 第12堂 | 進階實務：Vibe Coding 與 Agent 系統協作

學習目標

完成本堂課後，學員將能夠：

- 設計並實作**多 Agent 協作系統**，讓多個 Agent 分工完成複雜任務
- 建立一個 AI 全球新聞情報收集系統，整合跨語系資料
- 使用 AI 對異質資料進行趨勢分析
- 將分析結果以視覺化圖表方式呈現

核心概念說明

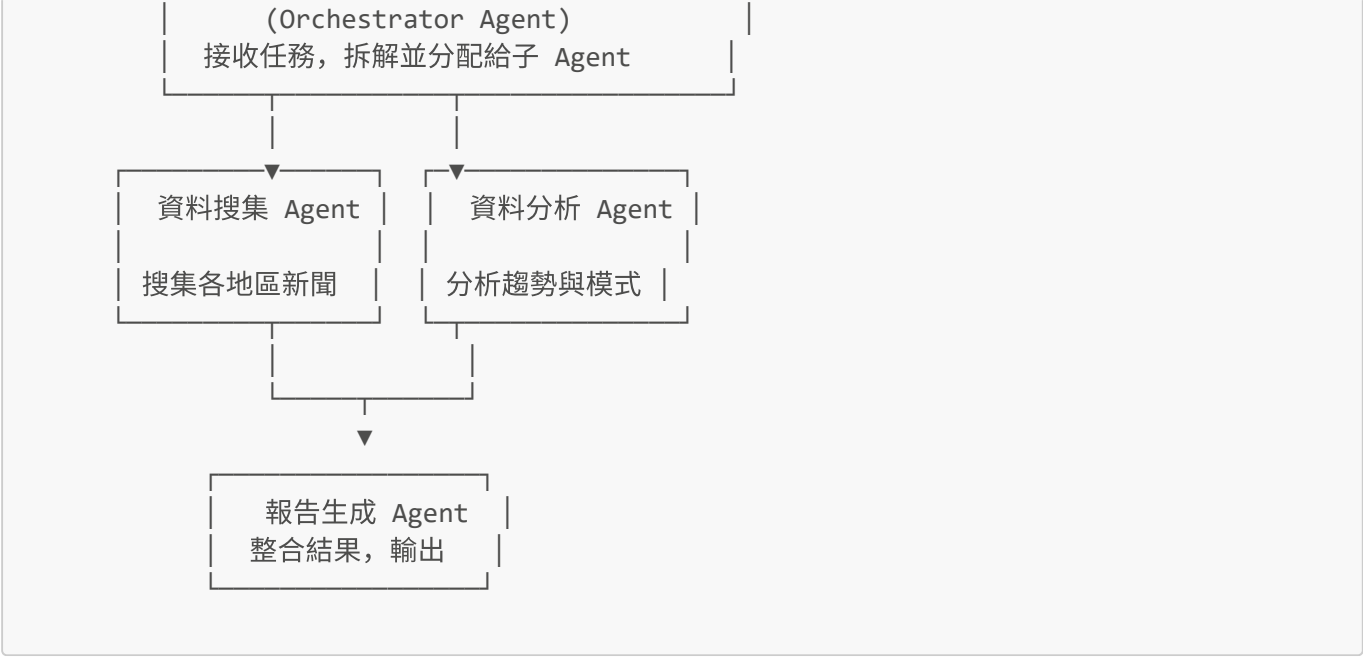
1. 多 Agent 協作系統

單一 Agent vs. 多 Agent 系統

	單一 Agent	多 Agent 系統
適合場景	單一明確的線性任務	複雜、多面向的任務
優點	設計簡單、好維護	可平行處理、分工明確
缺點	複雜任務時上下文過長	協調邏輯較複雜

多 Agent 協作的典型架構：





2. 全球新聞情報系統的需求分析

系統目標：

每天自動收集全球主要媒體的新聞，依主題分類後，提供趨勢分析和異常訊號偵測。

系統需要解決的挑戰：

挑戰	說明	解決方案
多語言	英文、中文、日文等不同語言	使用多語言 LLM 或翻譯 API
資料量大	每天可能有數百篇新聞	分批處理 + 重要性過濾
去重複	同一事件被多家媒體報導	語意相似度過濾
趨勢識別	找出突然增加的話題	關鍵字頻率時序分析
來源可信度	不同媒體的可信程度不同	建立媒體可信度白名單

3. 跨語系資料整合

多語言處理的兩種策略：

策略 A：翻譯後統一處理

外語文章 → 翻譯 API (Google Translate / DeepL) → 翻譯成中文 → 統一分析

優點：分析邏輯簡單

缺點：翻譯費用、可能損失一些細節

策略 B：使用多語言 LLM 直接處理

外語文章 → 直接送給多語言 LLM 分析 → 要求 LLM 以中文輸出結果
優點：保留語意精確度、省去翻譯步驟
缺點：需要 LLM 支援多語言能力（GPT-4o、Claude 均支援）

本課程採用策略 B（更簡單且效果好）：

Prompt 範例：
「以下是一篇英文/日文/韓文新聞，請你：
1. 判斷這篇新聞的主題類別
2. 提取關鍵資訊（誰、做了什麼、影響是什麼）
3. 給出一個 100 字以內的繁體中文摘要
4. 評估這則新聞的重要性（1-5分）和可信度（高/中/低）
請以 JSON 格式回傳。」

4. 趨勢分析的基本方法

什麼是「趨勢」？

趨勢分析的目的是回答：「關於某個主題，最近的討論是增加還是減少？有沒有突然的爆發？」

簡單的趨勢分析方法：

方法	說明	適用情境
關鍵字頻率統計	計算特定關鍵字在不同時間段出現的次數	追蹤熱門話題的消長
新聞數量時序圖	以折線圖呈現每日/每週的報導數量	識別討論高峰
情感趨勢	追蹤特定主題的情感從正面變負面或反之	品牌聲譽監控
異常偵測	找出短時間內大幅增加的主題	快速識別突發事件

趨勢視覺化的常用圖表：

圖表類型	適合展示
折線圖	隨時間的數量變化
熱力圖	主題 × 時間的強度分佈
詞雲	高頻關鍵字的視覺化
長條圖	各主題的報導量比較

5. 課程進階技術整合

本堂課是整個課程前半段的綜合應用，將整合以下技術：

技術來源

整合應用

第04堂 (Web爬蟲)	→	自動抓取各媒體網站新聞
第07堂 (API整合)	→	NewsAPI + 翻譯 API 資料取得
第03堂 (資料處理)	→	多語言新聞彙整與去重
第09堂 (情報分析)	→	AI 主題分類與重要性評估
第10堂 (視覺化)	→	趨勢圖表生成

實作步驟 (逐步教學)

實作一：設計多 Agent 協作架構

步驟 1：向 GitHub Copilot 描述協調者 Agent

請幫我設計一個多 Agent 協作的新聞情報系統，包含以下三個 Agent：

Agent 1 - 資料搜集 Agent：

- 從 NewsAPI 搜集英文、德文、日文新聞
- 每個語言最多取 30 篇
- 回傳標準化的新聞清單 (JSON)

Agent 2 - 分析 Agent：

- 輸入：新聞清單
- 對每篇新聞：分類主題、提取重點、輸出中文摘要、評估重要性
- 去除重複報導 (同一事件的不同媒體報導)
- 回傳分析後的結構化清單

Agent 3 - 報告 Agent：

- 輸入：分析後的清單
- 生成主題分佈統計
- 生成今日重要新聞摘要 (Top 5)
- 輸出 Excel 報告和趨勢圖表

協調者邏輯：

- 依序呼叫三個 Agent
- 處理每個 Agent 可能的失敗
- 紀錄整個流程的執行時間

實作二：建立趨勢分析模組

步驟 1：設計趨勢分析函式

請幫我建立一個新聞趨勢分析函式：

1. 輸入：過去 7 天的新聞分析結果 (每天一個 JSON 檔案)
2. 計算每個主題類別每天的報導數量

3. 識別「異常增加」的主題（今日數量 > 過去7日平均值的 2 倍）
4. 生成以下視覺化圖表（儲存為 PNG）：
 - a. 折線圖：各主題類別過去7天的報導量趨勢
 - b. 長條圖：今日各主題的報導數量比較
 - c. 詞雲：今日所有新聞標題的高頻關鍵字
5. 輸出一份「趨勢摘要」JSON，包含：
 - 今日最熱門主題 Top 3
 - 異常增加的主題清單
 - 與昨日相比的整體變化

步驟 2：安裝視覺化套件

```
pip install matplotlib
pip install wordcloud
pip install jieba # 中文分詞（用來正確切分中文詞彙）
```

實作三：建立排程執行機制

步驟 1：設定定時執行

請幫我建立一個排程腳本：

使用 Python 的 schedule 套件，讓系統每天早上 7:00 自動執行：

1. 搜集昨天的新聞資料
2. 執行分析
3. 生成報告和圖表
4. 透過 LINE Notify 發送今日重要新聞摘要 (Top 3)
5. 記錄執行結果到 log 檔案

請也加入以下功能：

- 如果某天執行失敗，發送失敗通知
- 保留最多 30 天的歷史資料
- 超過 30 天的舊資料自動打包壓縮後封存

程式碼範例（虛擬碼）

協調者 Agent

```
函式 新聞情報系統主流程():
    開始時間 = 記錄現在時間()
    顯示 "=== 開始執行全球新聞情報系統 ==="

    # Agent 1: 搜集資料
    顯示 "Step 1/3: 搜集新聞資料..."
    嘗試:
```

```

    原始新聞 = 資料搜集Agent(
        語言清單=["en", "de", "ja"],
        每語言篇數=30,
        關鍵字清單=["AI", "semiconductor", "trade", "energy"]
    )
    顯示 f"搜集完成：共 {len(原始新聞)} 篇"
如果發生錯誤：
    記錄錯誤("資料搜集 Agent 失敗")
    發送失敗通知("資料搜集失敗，請手動確認")
    結束

# Agent 2：分析資料
顯示 "Step 2/3：分析新聞內容..."
嘗試：
    分析結果 = 分析Agent(原始新聞)
    顯示 f"分析完成：{len(分析結果)} 篇新聞（已去重複）"
如果發生錯誤：
    記錄錯誤("分析 Agent 失敗")
    # 分析失敗時，儲存原始資料供後續手動處理
    儲存(原始新聞, "待分析原始資料.json")
    結束

# Agent 3：生成報告
顯示 "Step 3/3：生成報告..."
報告路徑 = 報告Agent(分析結果)

# 執行完成
執行時間 = 計算執行時間(開始時間)
顯示 f"=== 完成！耗時 {執行時間} 秒 ==="
顯示 f"報告路徑：{報告路徑}"

# 發送每日摘要
摘要 = 生成每日摘要(分析結果)
發送LINE通知(摘要)

```

趨勢分析與視覺化

載入 pandas, matplotlib, collections 套件

函式 生成趨勢圖(7天新聞資料):

```

# 整理每天每主題的數量
趨勢資料 = {}
對 每天 在 7天新聞資料 中:
    日期 = 每天["日期"]
    對 每篇新聞 在 每天["新聞"] 中:
        主題 = 每篇新聞["主題"]
        如果 主題 不在 趨勢資料 中:
            趨勢資料[主題] = {}
        趨勢資料[主題][日期] = 趨勢資料[主題].get(日期, 0) + 1

# 繪製折線圖

```

```

圖, 軸 = 建立圖表(大小=(14, 7))
對 每個主題, 資料 在 趨勢資料.items():
    日期清單 = 排序(資料.keys())
    數量清單 = [資料[日期] for 日期 in 日期清單]
    軸.畫折線(日期清單, 數量清單, 標籤=主題, 線條粗細=2)

軸.設定標題("過去7天主題報導量趨勢")
軸.設定X軸標題("日期")
軸.設定Y軸標題("報導篇數")
軸.顯示圖例()
軸.旋轉X軸標籤(45度)
儲存圖表("趨勢折線圖.png", dpi=150, 緊密排版=True)

# 識別異常增加的主題
異常主題 = []
對 每個主題 在 趨勢資料 中:
    今日數量 = 今天的數量(趨勢資料[主題])
    過去6日平均 = 計算平均(過去6天的數量)
    如果 今日數量 > 過去6日平均 * 2:
        異常主題.加入({
            "主題": 主題,
            "今日": 今日數量,
            "平均": 過去6日平均,
            "增幅": f"{{(今日數量/過去6日平均 - 1)*100:.0f}}%"
        })

回傳 異常主題

```

練習題 / 作業

課堂練習

練習 1: Agent 分工設計

針對以下複雜任務，設計一個多 Agent 系統（描述每個 Agent 的職責和輸入/輸出）：

任務：「每週自動生成一份競品分析報告，包含主要競爭對手的最新動態、定價變化、和市場輿論」

Agent 名稱	職責	輸入	輸出

練習 2: 趨勢圖解讀

假設你看到以下趨勢：「AI 晶片」這個主題在過去 7 天的報導量：

週一：5篇、週二：4篇、週三：6篇、週四：5篇、週五：4篇、週六：5篇、**週日：23篇**

- 這意味著什麼？
- 作為企業決策者，你的下一步行動是什麼？
- 如何設計 Alert 機制讓系統自動通知你？

課後作業

1. **執行完整系統：**在自己的環境中執行課堂建立的新聞情報系統，搜集並分析一天的新聞，觀察輸出的報告品質，記錄至少 3 個可以改進的地方。
 2. **趨勢分析設計：**設計一個「競品監控」版本的趨勢分析，指定你的主要競爭對手（可以是任何行業），說明要追蹤哪些關鍵字、多久執行一次、異常觸發的標準是什麼。
-

延伸閱讀

多 Agent 框架：

- LangGraph — LangChain 的多 Agent 協作框架 — <https://langchain-ai.github.io/langgraph/>
- AutoGen (Microsoft) — 多 Agent 對話框架 — <https://microsoft.github.io/autogen/>

資料視覺化：

- matplotlib 官方文件 — <https://matplotlib.org/stable/gallery/>（有大量範例）
- seaborn — 更美觀的統計圖表 — <https://seaborn.pydata.org/>
- wordcloud 套件 — https://github.com/amueller/word_cloud

排程執行：

- schedule 套件 — <https://schedule.readthedocs.io/>
 - APScheduler — 更強大的排程工具 — <https://apscheduler.readthedocs.io/>
-

10/13 | 第13堂 | 案例分析：企業供應鏈數據管理與結構化工程

學習目標

完成本堂課後，學員將能夠：

- 分析企業供應鏈數據管理的實際痛點與自動化機會
 - 從格式不一的廠商報價單中自動提取關鍵資訊
 - 建立自動化的報價單歸檔與版本管理系統
 - 實作 KPI 指標的自動化計算與多廠商比對功能
-

核心概念說明

1. 企業供應鏈數據管理的痛點

典型企業場景：

一家製造業公司，每次採購物料時需要：

1. 向 10-20 家廠商詢價
2. 各廠商用自己的格式回傳報價單（Excel、PDF、Word 都有）
3. 採購人員逐一打開報價單，手動複製關鍵資料到比較表
4. 計算各廠商的綜合評分（價格、交期、品質等）

5. 製作比較報告提交主管審核

這個流程的問題：

問題	影響
格式不統一	每份報價單都要重新閱讀理解結構
手動複製資料	容易出現輸入錯誤
比較邏輯複雜	要同時考慮多個維度，容易遺漏
歸檔不系統	找歷史報價困難，版本管理混亂
耗時	採購人員的大量時間花在這些低附加價值的工作

2. 異質資料提取的挑戰

為什麼廠商的報價單格式不統一？

- 各廠商有自己的 ERP 系統，匯出格式各異
- 有些廠商習慣用 Excel 模板，表格結構不同
- 有些廠商只用 PDF，甚至是掃描紙本
- 欄位名稱不一樣（「單價」vs「每件價格」vs「Unit Price」）
- 計算邏輯不同（含稅/不含稅、是否包含運費）

AI 在異質資料提取的優勢：

傳統的程式處理異質資料需要為每種格式寫獨立的解析邏輯（維護成本極高），而 LLM 能夠理解各種格式的語意，直接提取結果。

3. KPI 指標設計

比較廠商時，通常不能只看價格，需要考慮多個維度：

採購 KPI 評估框架（範例）：

KPI 指標	說明	權重（可自訂）
單價	每件/每批的價格	40%
交期	從下單到交貨的天數	25%
MOQ	最低訂購量（影響彈性）	10%
付款條件	幾天內付款，是否有折扣	10%
品質等級	ISO 認證、過去的不良率	15%

KPI 加權評分計算：

廠商總分 =
(單價分數 × 40%) +
(交期分數 × 25%) +
(MOQ分數 × 10%) +
(付款條件分數 × 10%) +
(品質等級分數 × 15%)

各指標的分數計算：

單價分數 = $100 \times (\text{最低單價} / \text{該廠商單價})$

交期分數 = $100 \times (\text{最短交期} / \text{該廠商交期})$

...

4. 報價單歸檔系統設計

一個好的報價單歸檔系統需要：

資料夾結構設計：

```
/供應商報價管理/  
  / [廠商名稱] /  
    / [詢價品項] /  
      / [年份] /  
        原始報價單_2025Q3_V1.pdf  
        原始報價單_2025Q3_V2.xlsx ← 廠商修改後的版本  
        解析結果_2025Q3.json ← AI 提取的結構化資料  
        比較報表_2025Q3.xlsx ← 最終的比較結果
```

版本管理的重要性：

廠商有時會在報價後調整價格，版本管理讓你能：

- 追蹤哪個版本的報價最終被採用
- 記錄廠商的報價歷史，用於下次議價的依據
- 稽核時提供完整的報價紀錄

5. 供應鏈場景的特殊考量

在供應鏈場景中，AI 自動化需要特別注意：

資料準確性要求極高：

風險	說明	控制措施
AI 提取錯誤	金額、數量提取有誤	所有提取結果需人工確認
匯率換算	外幣報價需要正確換算	匯率來源要可追溯

風險	說明	控制措施
幣別混淆	同一份報價單可能混用幣別	明確標記每個金額的幣別
單位不統一	kg vs g、pcs vs 箱	統一換算為標準單位

實作步驟（逐步教學）

實作一：報價單資料提取引擎

步驟 1：準備測試資料

建立 3-5 份格式不同的模擬報價單（可以用 Excel 或 Word 手動建立），確保它們有不同的結構但包含相同類型的資訊：

- 廠商名稱與聯絡資訊
- 報價品項（品名、規格、數量、單價）
- 交期、付款條件、有效期限
- 備註事項

步驟 2：向 GitHub Copilot 描述提取引擎

```
請幫我建立一個報價單資料提取系統：

1. 支援的輸入格式：
- Excel (.xlsx, .xls): 使用 pandas 讀取
- PDF (.pdf): 使用 pdfplumber 提取文字
- Word (.docx): 使用 python-docx 提取文字

2. 對提取的內容，呼叫 LLM 進行結構化提取，回傳 JSON：
{
  "廠商資訊": {
    "廠商名稱": "",
    "聯絡人": "",
    "Email": "",
    "電話": ""
  },
  "報價日期": "",
  "有效期限": "",
  "幣別": "TWD/USD/EUR",
  "品項": [
    {
      "品名": "",
      "規格": "",
      "數量": 0,
      "單位": "",
      "單價": 0,
      "小計": 0
    }
  ],
  "交期": "",
```

```
"付款條件": "",  
"是否含稅": true/false,  
"備註": "",  
"AI信心度": "高/中/低" ← AI 對提取結果準確性的自我評估  
}
```

3. 若 AI 信心度為「低」，在結果中標記 "需人工確認": true

實作二：多廠商比較與 KPI 評分

步驟 1：設計 KPI 評分函式

請幫我建立一個多廠商 KPI 比較系統：

1. 輸入：多個廠商的結構化報價資料（字典清單）
2. 可自訂的 KPI 權重設定（從 config.json 讀取）
3. 計算每個廠商的各指標分數和加權總分
4. 輸出一份 Excel 比較表，包含：
 - 第一頁「廠商比較總覽」：所有廠商並排比較，最低價格和最高分數自動標綠色
 - 第二頁「品項詳細比較」：每個品項的各廠商單價比較
 - 第三頁「KPI 雷達圖」：視覺化各廠商在不同維度的表現
 - 第四頁「採購建議」：AI 根據數據給出的採購建議
5. 加入以下保護機制：
 - 所有貨幣自動換算為台幣（匯率從環境變數讀取）
 - 交期如果是「4-6週」、「30 天」等不同格式，統一換算為天數
 - 如果某廠商的某個欄位是 "UNKNOWN"，在比較表中特別標記

實作三：自動歸檔系統

步驟 1：建立歸檔流程

請幫我建立一個報價單自動歸檔系統：

1. 監控「待處理報價單」資料夾（使用 watchdog）
2. 偵測到新檔案時：
 - a. 提取報價資料（呼叫實作一的函式）
 - b. 自動建立歸檔資料夾：/報價管理/{廠商名稱}/{詢價品項}/{年季}/
 - c. 將原始檔案複製到歸檔資料夾（自動加上版本號：_v1, _v2...）
 - d. 將提取結果存為 JSON 檔案
 - e. 更新「報價總覽」Excel（所有廠商所有批次的彙整表）
3. 每次新增報價時，自動：
 - a. 與同批次的其他廠商報價比較
 - b. 如果比最低價高超過 20%，發出 "價格偏高" 提醒
 - c. 若達到 3 份以上報價，自動生成初步比較報告

程式碼範例（虛擬碼）

報價單資料提取

載入 pdfplumber, pandas, docx, openai, json 套件

函式 讀取檔案內容(檔案路徑):

```
副檔名 = 取得副檔名(檔案路徑).轉小寫()
```

```
如果 副檔名 在 [".xlsx", ".xls"]:
```

```
    資料 = pandas.讀取Excel(檔案路徑)
```

```
    回傳 資料.轉為字串() # 將表格轉為文字
```

```
否則如果 副檔名 == ".pdf":
```

```
    使用 pdfplumber.open(檔案路徑) 作為 pdf:
```

```
        全文 = ""
```

```
        對 每頁 在 pdf.pages 中:
```

```
            全文 += 每頁.extract_text()
```

```
        回傳 全文
```

```
否則如果 副檔名 == ".docx":
```

```
    文件 = docx.Document(檔案路徑)
```

```
    全文 = "\n".join([段落.text for 段落 in 文件.paragraphs])
```

```
    回傳 全文
```

```
否則:
```

```
    拋出錯誤(f"不支援的檔案格式: {副檔名}")
```

函式 提取報價資料(檔案路徑):

```
# 讀取檔案
```

```
原始內容 = 讀取檔案內容(檔案路徑)
```

```
# AI 結構化提取
```

```
提示詞 = f"""
```

```
以下是一份廠商報價單的內容:
```

```
{原始內容}
```

請提取所有的報價資訊，以 JSON 格式回傳。
格式如下：[JSON Schema 定義]

特別注意：

- 若資訊不明確，填入 "UNKNOWN" 並將 AI信心度 設為「低」
- 金額填入數字（不含貨幣符號）
- 日期統一格式為 YYYY-MM-DD

只回傳 JSON，不要說明。

"""

回應 = 呼叫LLM(提示詞, 格式=JSON模式)

結果 = 解析JSON(回應)

結果["來源檔案"] = 檔案路徑

結果["提取時間"] = 現在時間()

標記需人工確認的欄位

如果 結果["AI信心度"] == "低":

 結果["需人工確認"] = 真

回傳 結果

KPI 評分計算

函式 計算KPI評分(廠商報價清單, KPI權重設定):

 評分結果 = []

計算各指標的最佳值（用來算相對分數）

 最低單價 = 找最小值([廠商["品項"][0]["單價"] for 廠商 in 廠商報價清單])

 最短交期 = 找最小值([轉換為天數(廠商["交期"]) for 廠商 in 廠商報價清單])

對 每個廠商 在 廠商報價清單 中:

 各項分數 = {}

 # 單價分數（越低越好）

 廠商單價 = 廠商["品項"][0]["單價"]

 各項分數["單價"] = (最低單價 / 廠商單價) * 100

 # 交期分數（越短越好）

 廠商交期 = 轉換為天數(廠商["交期"])

 各項分數["交期"] = (最短交期 / 廠商交期) * 100

加權計算總分

 總分 = 0

 對 每個KPI, 分數 在 各項分數.items():

 權重 = KPI權重設定.get(KPI, 0)

 總分 += 分數 * 權重 / 100

 評分結果.加入({

 "廠商名稱": 廠商["廠商資訊"]["廠商名稱"],

 "各項分數": 各項分數,

 "加權總分": round(總分, 2),

```
"推薦": 總分 >= 80 # 80分以上標記為推薦
})
```

```
# 依總分排序
回傳 排序(評分結果, 依照="加權總分", 降序=真)
```

練習題 / 作業

課堂練習

練習 1: KPI 權重設計

針對以下三種採購情境，設計不同的 KPI 權重（各項加起來等於 100%）：

KPI 指標	緊急採購（週內需要）	大批量採購（年度合約）	高品質零件採購
單價			
交期			
MOQ			
付款條件			
品質等級			

練習 2: 資料提取挑戰識別

準備一份你在工作中實際收到過的複雜文件（可以用模擬資料），思考：

- AI 在提取哪些欄位時最容易出錯？
- 哪些欄位的格式多樣性最高？
- 如何設計「信心度」的評估邏輯？

課後作業

1. **完整端到端測試：**準備 3 份格式不同的模擬報價單（建議用不同的 Excel 格式，或一個 Excel + 一個 PDF），執行完整的提取→評分→比較流程，評估輸出的準確性。
2. **歸檔系統設計：**根據課堂討論的概念，設計你的工作場景中的文件歸檔系統結構，說明：資料夾層級設計、命名規則、版本管理策略。

延伸閱讀

供應鏈相關：

- 《供應鏈管理》— 了解採購管理的基本概念
- APICS 供應鏈專業知識 — 業界標準的供應鏈管理框架

多格式文件處理：

- python-docx — <https://python-docx.readthedocs.io/>
- pdfplumber — <https://github.com/jsvine/pdfplumber>
- Python xlrd（讀取舊版 .xls 格式） — <https://xlrd.readthedocs.io/>

資料驗證：

- Pydantic — Python 資料驗證框架 — <https://docs.pydantic.dev/>（可用來驗證 AI 提取結果的格式是否正確）

10/20 | 第14堂 | 案例分析：敏捷管理自動化—智慧週報生成系統

學習目標

完成本堂課後，學員將能夠：

- 了解敏捷（Agile）專案管理的基本概念與資料結構
- 整合來自多個來源的跨專案工作資料
- 使用 AI 從原始工作紀錄中自動提取關鍵里程碑
- 將原始工作資料轉化為具備管理深度的 PPT 決策報表

核心概念說明

1. 敏捷管理的週報痛點

典型情境：

PM（專案經理）每週五要整理週報，流程通常是：

1. 從 Jira 匯出本週的任務清單（CSV）
2. 打開 Excel 手動計算完成率、延遲任務數、風險項目
3. 詢問各組組長本週的主要成果
4. 整理以上資訊，寫成 Word 報告或 PowerPoint
5. 每週重複這個 2-3 小時的工作

企業的實際痛點：

問題	影響
資料散落各處	需要從多個系統蒐集，費時且易遺漏
格式不一致	不同組別的周報格式不同，彙整困難
分析深度不足	時間緊迫下，大多是數字羅列，缺乏洞察
版本管理問題	找上週的報告耗時
高重複性工作	PM 寶貴的時間被這類工作佔據

2. 敏捷管理的核心資料結構

要自動化週報生成，需要理解敏捷管理的基本資料：

Sprint（衝刺）：

- 敏捷開發的基本工作週期，通常 1-2 週
- 每個 Sprint 有明確的開始和結束日期
- Sprint 結束後要達成一個「可交付的成果」

Story / Task：

- Sprint 中的工作任務
- 每個任務有：標題、負責人、狀態、預估工時、實際工時、優先級

里程碑（Milestone）：

- 專案的重要轉折點或交付節點
- 不是每週都有，但需要在報告中特別標注

KPI 指標：

指標	計算方式	說明
Sprint 完成率	完成任務數 / 計劃任務數	衡量計畫執行能力
工時燃盡率	已用工時 / 計畫工時	判斷是否超支
Bug 密度	發現的 Bug 數 / 完成的功能數	衡量品質
阻塞任務比例	阻塞中任務數 / 總任務數	識別風險

3. 跨專案資料整合架構

當公司有多個並行專案時，週報需要呈現所有專案的整合視圖：

資料來源可能包含：

資料來源 1: Jira / Trello / Asana

- └─ 任務清單 (CSV 或 API)

資料來源 2: Git (GitHub / GitLab)

- └─ Commit 記錄 (代碼提交歷史)

資料來源 3: 日常工作紀錄

- └─ 會議記錄 (Word / Notion)
- └─ 組員的週記 (純文字)

資料來源 4: 時程表

- └─ 原始計畫甘特圖 (Excel)

整合挑戰：

- 不同系統的任務 ID 不相同
- 同一個任務在不同系統中可能有不同的名稱
- 時區不一致
- 部分系統可能暫時無法連接

4. AI 提取里程碑的邏輯

原始工作紀錄通常是這樣的文字：

「本週完成了登入模組的單元測試，所有 32 個測試案例通過。前端 UI 設計稿已獲客戶確認，可以開始實作。API 文件初版完成，已送審。」

AI 需要從這段文字中識別：

- **里程碑事件：**「所有 32 個測試案例通過」→ 測試完成里程碑
- **待追蹤事項：**「已送審」→ 有待確認的項目
- **相依關係：**「已獲客戶確認，可以開始實作」→ 解鎖了下一個任務

AI 識別里程碑的 Prompt 設計：

你是一位資深的 PM，請從以下工作紀錄中識別本週的關鍵里程碑。

里程碑的定義：

1. 重要功能完成並通過驗收
2. 關鍵決策已獲批准
3. 重要的外部依賴已解決
4. 階段性交付物完成

對於每個里程碑，請說明：

- 里程碑名稱（簡潔的標題）
- 達成日期
- 影響（對專案的意義）
- 下一步行動

工作紀錄：[內容]

5. PPT 決策報表設計原則

面向管理層的 PPT 報告，與面向執行層的工作報告截然不同：

管理層關注的問題：

- 我們本週有沒有達成目標？
- 哪裡有風險需要我關注？
- 有沒有需要我做決定的事項？

PPT 簡報頁面規劃：

頁碼	頁面標題	內容
1	封面	週報標題、週次、報告日期
2	本週整體健康度	交通燈狀態（紅/黃/綠）+ 一句話摘要
3	關鍵里程碑	本週達成 + 下週預計
4	各專案 KPI 儀表板	完成率、工時燃盡、品質指標的視覺化
5	風險與阻塞項目	需要管理層關注的問題清單
6	需要決策的事項	具體的決策請求，附上選項與建議
7	下週計畫	各專案下週的主要目標

實作步驟（逐步教學）

實作一：資料整合模組

步驟 1：設計通用的資料讀取介面

請幫我建立一個跨來源的工作資料整合模組：

支援以下資料來源：

- CSV 檔案（Jira/Trello 匯出格式）：
 - 自動偵測欄位名稱（不同系統欄位名稱不同）
 - 使用 LLM 進行欄位名稱的語意映射
（例如 "Status"、"狀態"、"state" 都映射到統一的 "task_status"）
- 工作紀錄文字檔（.txt 或 .docx）：
 - 支援多種結構（日記格式、條列式、會議記錄格式）
 - 使用 LLM 提取結構化的任務資訊
- Excel 時程表：
 - 讀取計畫日期 vs 實際日期

輸出統一的資料格式：

```
{  
  "任務ID": "唯一識別碼",  
  "任務名稱": "",  
  "所屬專案": "",  
  "負責人": "",  
  "狀態": "未開始/進行中/已完成/阻塞",  
  "計畫開始日": "YYYY-MM-DD",  
  "計畫結束日": "YYYY-MM-DD",  
  "實際開始日": "YYYY-MM-DD",  
  "實際結束日": "YYYY-MM-DD",  
  "優先級": "高/中/低",  
  "備註": ""  
}
```

實作二：AI 里程碑提取與分析

步驟 1：建立里程碑提取引擎

在 Copilot Chat 中輸入：

請建立一個 `sprint_analyzer.py` 程式，功能如下：

1. 輸入：整合後的任務清單（上一個步驟的輸出）+ 本週工作紀錄文字
2. 計算 Sprint KPI：
 - Sprint 完成率 = 狀態為"已完成"的任務數 / 總任務數
 - 按時完成率 = 實際結束日 <= 計畫結束日 的比例
 - 阻塞率 = 狀態為"阻塞"的任務比例
 - 工時使用率（如果有工時資料）
3. 用 LLM 分析以下問題：
 - Q1：本週有哪些關鍵里程碑？（里程碑名稱、意義、影響）
 - Q2：有哪些風險或阻塞需要管理層關注？
 - Q3：哪些任務有延遲，延遲原因是什麼？
 - Q4：下週最重要的 3 件事是什麼？
 - Q5：有沒有需要管理層決策的事項？
4. 輸出一個結構化的分析結果字典，供後續 PPT 生成使用

實作三：週報 PPT 自動生成

步驟 1：設計 PPT 生成邏輯

請建立一個 `report_generator.py`，使用 `python-pptx` 生成週報 PPT：

1. 載入 PPT 模板（事先設計好的 `.pptx` 檔，有正確的字型、配色和 Logo）
2. 生成以下頁面：

封面頁：

- 標題："{公司名稱} 專案週報"
- 副標題："第 {週次} 週 | {日期範圍}"

整體健康度頁：

- 大圓形指示燈：綠（良好） / 黃（需注意） / 紅（有風險）
- 顏色判斷規則：
 - * 綠色：Sprint 完成率 $\geq 80\%$ 且 阻塞率 $\leq 10\%$
 - * 黃色：Sprint 完成率 60-79% 或 阻塞率 11-20%
 - * 紅色：Sprint 完成率 $< 60\%$ 或 阻塞率 $> 20\%$
- 一句話摘要（由 LLM 生成）

KPI 儀表板頁：

- 對每個專案，顯示一個條狀圖（Sprint 完成率）

- 完成率低於 70% 的標記為紅色

里程碑頁：

- 表格：本週達成的里程碑（里程碑名稱、達成日、重要性）
- 表格：下週預計達成的里程碑

風險與決策頁：

- 風險清單（條列式）
- 需決策事項（條列式，每項包含：問題描述、建議選項、截止日期）

3. 儲存 PPT 為："{公司名稱}_週報_w{週次}_{日期}.pptx"

實作四：自動化排程

步驟 1：設定每週自動執行

請建立 scheduler.py，設定每週五下午 4 點自動執行完整的週報生成流程：

1. 從指定資料夾讀取本週的所有資料來源檔案
2. 執行整合 → 分析 → 生成 PPT
3. 將生成的 PPT 儲存到「週報歸檔」資料夾
4. 發送郵件通知相關人員（使用第 5 堂的 Outlook 自動化）
5. 在 console 顯示執行摘要（哪些步驟成功/失敗）

使用 Python 的 schedule 套件設定排程：

```
import schedule
schedule.every().friday.at("16:00").do(執行週報流程)
```

程式碼範例（虛擬碼）

里程碑提取

函式 提取里程碑(任務清單，工作紀錄文字)：

提示詞 = f"""

你是一位資深 PM，請根據以下資訊提取本周關鍵里程碑。

任務狀態變更（本週從進行中變為已完成的任務）：

{過濾出本週完成的任務(任務清單)}

工作紀錄文字：

{工作紀錄文字}

請輸出 JSON 格式，包含：

```
[
  {{
    "名稱": "里程碑簡短標題",
    "達成日期": "YYYY-MM-DD",
```

```

        "描述": "一句話說明這個里程碑的意義",
        "影響": "對整體專案進度的影響",
        "後續行動": "這個里程碑解鎖了什麼下一步"
    }}
]
"""

```

```

LLM回應 = 呼叫LLM(提示詞, 格式=JSON模式)
里程碑清單 = 解析JSON(LLM回應)

```

```

回傳 里程碑清單

```

函式 生成管理層摘要(KPI數據, 里程碑清單, 風險清單):

```

提示詞 = f"""

```

```

你是一位高階 PM, 需要向 CXO 報告本週專案狀況。

```

```

KPI 數據:

```

```

{KPI數據}

```

```

關鍵里程碑:

```

```

{里程碑清單}

```

```

風險事項:

```

```

{風險清單}

```

```

請生成一段 3-5 句的執行摘要, 使用管理層的语言:

```

- 不要用技術術語
- 強調業務影響而非技術細節
- 清楚指出需要關注的重點

```

"""

```

```

摘要 = 呼叫LLM(提示詞)

```

```

回傳 摘要

```

PPT 生成

```

載入 python_pptx 套件

```

函式 生成週報PPT(分析結果, 模板路徑, 輸出路徑):

```

簡報 = 開啟簡報(模板路徑)

```

```

# 封面頁 (模板第 0 頁)

```

```

封面頁 = 簡報.slides[0]

```

```

設定文字框(封面頁, "標題佔位符", f"專案週報 w{分析結果['週次']}")

```

```

設定文字框(封面頁, "日期佔位符", 分析結果["日期範圍"])

```

```

# 整體健康度頁 (模板第 1 頁)

```

```

健康度頁 = 簡報.slides[1]

```

```

顏色 = 判斷健康度顏色(分析結果["KPI"])

```

```

設定圓形顏色(健康度頁, 顏色)

```

```

設定文字框(健康度頁, "摘要佔位符", 分析結果["管理層摘要"])

```

```

# KPI 儀表板頁 (模板第 2 頁)
KPI頁 = 簡報.slides[2]
對 每個專案 在 分析結果["各專案KPI"] 中:
    顏色 = "紅色" 如果 專案["完成率"] < 70 否則 "綠色"
    新增條狀圖(KPI頁, 專案名稱=專案["名稱"], 完成率=專案["完成率"], 顏色=顏色)

# 里程碑頁 (模板第 3 頁)
里程碑頁 = 簡報.slides[3]
里程碑表格資料 = [
    ["里程碑", "達成日期", "重要性"],
    *[["m["名稱"], m["達成日期"], m["描述"]] for m in 分析結果["里程碑"]]
]
新增表格(里程碑頁, 里程碑表格資料)

# 風險與決策頁 (模板第 4 頁)
風險頁 = 簡報.slides[4]
對 i, 風險 在 enumerate(分析結果["風險清單"]):
    新增條列(風險頁, i, f"△ {風險}")
對 i, 決策 在 enumerate(分析結果["決策事項"]):
    新增條列(風險頁, i + 10, f"🌀 {決策['問題']}\n    建議: {決策['建議']}")

簡報.save(輸出路徑)
顯示("✅ 週報已生成:" + 輸出路徑)
回傳 輸出路徑

函式 判斷健康度顏色(KPI):
    完成率 = KPI["sprint完成率"]
    阻塞率 = KPI["阻塞率"]

    如果 完成率 >= 80 且 阻塞率 <= 10:
        回傳 "綠色"
    否則如果 完成率 >= 60 且 阻塞率 <= 20:
        回傳 "黃色"
    否則:
        回傳 "紅色"

```

練習題 / 作業

課堂練習

練習 1: 分析工作紀錄文字

請閱讀以下模擬的工作紀錄，練習手動識別哪些句子代表「里程碑」：

「本週完成後台管理介面的開發，已合併到主分支。客戶驗收測試計畫已與客戶確認，排定下週一開始。發現了一個影響報表生成的嚴重 Bug，已修復並部署到測試環境。資料庫遷移腳本已完成撰寫，待 DBA 審核後執行。行銷活動的 Landing Page 因設計變更，將延遲一週交付。與第三方金流供應商的 API 整合已完成測試，準備提交正式環境申請。」

請識別：

1. 哪些是本週達成的里程碑（具體、已完成的成果）？
2. 哪些是需要追蹤的待確認事項？
3. 哪些是需要管理層關注的風險？

練習 2：PPT 頁面規劃

若你是你目前工作部門的 PM，設計一份面向主管的週報 PPT 框架：

- 每週最重要的 3 個管理層關注指標是什麼？
- 主管最想在報告中得到什麼問題的答案？
- 哪些資訊應該放在 PPT，哪些只需要附件？

課後作業

整合作業（本課程結案作業）：

選擇你在本課程中開發的任何一個或多個系統，製作一份 5-7 頁的 PPT 成果展示：

1. **問題描述**（1 頁）：這個系統解決了什麼業務問題？
2. **系統架構**（1-2 頁）：
 - 整個自動化流程的資料流圖
 - 使用了哪些工具和技術
3. **成果展示**（2-3 頁）：
 - 執行前後的效率比較（時間節省、錯誤減少）
 - 一個實際的執行範例（輸入 → 輸出的截圖）
4. **困難與學習**（1 頁）：
 - 遇到了什麼挑戰？如何解決？
 - 下一步想要加入什麼功能？

延伸閱讀

敏捷管理相關：

- Scrum Guide — <https://scrumguides.org/>（Scrum 官方指南，可下載繁體中文版）
- 《敏捷革命》（The Agile Revolution）— 了解敏捷管理的起源與哲學

Python 工具：

- python-pptx 官方文件 — <https://python-pptx.readthedocs.io/>
- schedule 套件 — <https://schedule.readthedocs.io/>

生成式 AI 在企業應用：

- 《Building LLM Apps》— 實用的企業 AI 應用設計指南
- Microsoft Copilot for Microsoft 365 — 了解 AI 如何整合進企業工作流程